VVU Release Engineering Standard

VRES v1.0

A Complete Engineering Standard for
Reproducible, Verifiable, Auditable Software Releases

VVU Integrated Verification Environment

Document Date: 2026-08-06

Classification: Confidential

Document ID	VRES-1.0
Version	1.0.0
Status	Normative
Products	IVE, HBK Mk-II, ProofBridge, AIR Runtime, Ubuntu Pools, Trust Sphere
Operating Context	South African Municipal Water Infrastructure
Currency	ZAR (South African Rand)

Table of Contents

Volume I

Governance

Chapter 1 — Release Philosophy

1.1 Purpose and Scope

This standard defines the release engineering requirements for all VVU products. It establishes the mandatory principles, processes, roles, and artifacts that govern how software is built, tested, verified, signed, and distributed. Every VVU product — including IVE, HBK Mk-II, ProofBridge, AIR Runtime, Ubuntu Pools, and Trust Sphere — **MUST** comply with this standard as a condition of release. The standard is organized into twenty volumes covering the complete release lifecycle from initial development through end of life, with normative requirements expressed using RFC 2119 key words: **MUST**, **SHALL**, **MAY**, **SHOULD**, and their negations.

The scope of this standard extends to every artifact that is distributed outside the development environment, including but not limited to: compiled binaries, container images, firmware packages, installation media, documentation bundles, SBOM documents, and cryptographic provenance records. Internal development artifacts that are never distributed are not subject to the full standard but **SHOULD** follow its principles where practical.

1.2 Foundational Principles

1.2.1 Reproducibility

Every release build **MUST** be reproducible. Given the same source commit, dependency lockfile, and build configuration, any qualified builder **MUST** produce bit-identical output artifacts. Reproducibility is the foundation of trust in the release process: if two independent builders produce different outputs from the same inputs, the integrity of the entire pipeline is compromised. VVU achieves reproducibility through deterministic compilation (Volume III), dependency locking (Volume IV), and hermetic build environments that eliminate environmental variance such as timestamps, locale settings, and uninitialized memory. Where bit-identical reproducibility is technically infeasible (e.g., due to compiler-induced variance), the build **MUST** produce functionally equivalent output validated through a cryptographic manifest (Volume VII) that records every input and environmental parameter sufficient to reproduce the build outcome.

1.2.2 Determinism

Release processes **MUST** be deterministic. Given the same inputs, the same sequence of operations **MUST** produce the same outputs. Non-determinism in the release pipeline — whether from non-deterministic compilers, unordered file system traversal, randomized test shuffling, or time-dependent behavior — is a defect that **MUST** be eliminated. The build system **SHALL** enforce deterministic behavior through: (a) sorted file enumeration in all packaging and checksum operations, (b) fixed timestamps in archive headers (the SOURCE_DATE_EPOCH convention), (c) frozen dependency versions via lockfiles, and (d) hermetic toolchains that do not depend on the host environment. The vres doctor command (Section 23.3) **MUST** detect and report non-deterministic build conditions before any release candidate is promoted.

1.2.3 Traceability

Every released artifact **MUST** be traceable to its exact source commit, dependency versions, build configuration, builder identity, and verification results. Traceability is achieved through the audit framework (Volume XV), which creates an immutable record for every release containing: the Git SHA of the source tree, the exact dependency lockfile hash, the builder identity (verified via Sigstore), the compiler and tool versions, the complete SBOM, and the results of all quality gates (Volume XIII). This audit record **MUST** be retained for the lifetime of the product plus a minimum of seven years, and **MUST** be accessible for compliance audits without requiring access to the live build infrastructure.

1.2.4 Auditability

Every release **MUST** be independently auditable. An auditor who was not involved in the release process **MUST** be able to, using only the published artifacts and audit records: (a) verify that the claimed source commit produces the claimed binary artifacts, (b) confirm that all mandatory quality gates passed, (c) validate the SBOM against the actual dependency tree, (d) verify digital signatures on all signed artifacts, and (e) reproduce the complete audit trail from commit to release. The enterprise procurement pack (Volume XVII) **MUST** contain sufficient information for external auditors to perform this verification without access to VVU internal systems. Auditability is not optional — it is a core product requirement that directly enables enterprise procurement, government compliance, and academic reproducibility.

1.2.5 Engineering Ethics

VVU products operate in contexts where software integrity has real-world safety and financial consequences. HBK Mk-II operates in South African municipal water infrastructure where incorrect readings could affect public health. ProofBridge issues financial receipts where integrity failures could cause material harm. Ubuntu Pools manages community savings (stokvel) where cryptographic integrity is the basis of trust. For these reasons, the release process **MUST** uphold the following ethical principles: (a) No release **SHALL** be distributed with known defects that could cause harm in the product operating context. (b) No fabricated data **SHALL** be included in any release artifact — the zero fabrication rule is absolute. (c) Missing values **MUST** be explicitly marked as UNDEFINED, MISSING, REQUIRES VALIDATION, or PENDING — never inferred. (d) Security vulnerabilities identified before release **MUST** be resolved or documented in the release notes with remediation guidance. (e) The forbidden terms list (SAFE_FOR_DEPLOYMENT, Engineering certified, FEA verified, Physically validated, System safe) **MUST** never appear in any release artifact, as they imply guarantees that have not been formally verified.

1.3 Conformance

A VVU product conforms to this standard at level A if it satisfies all **MUST** and **SHALL** requirements. A product conforms at level B if it satisfies all **MUST** and **SHALL** requirements and all **SHOULD** requirements. Conformance claims **MUST** reference the specific version of this standard and the conformance level achieved. The vres lint command (Section 23.4) **MUST** report the conformance level of the product repository against this standard.

Chapter 2 — Release Lifecycle

2.1 Lifecycle States

Every VVU release progresses through the following states. Each state transition requires the mandatory gate for that transition to pass. A release **MUST NOT** advance past a state unless its gate has passed. The lifecycle is linear and unidirectional — a release does not move backward through states. If a gate fails, the release remains in its current state until the failure is remediated and the gate is re-evaluated. Hotfix releases follow an abbreviated lifecycle as specified in Volume XII, Section 12.7.

State #	State	Description	Entry Gate
1	Idea	Feature request accepted	Product owner approval
2	Development	Active implementation	Code review completion
3	Feature Freeze	No new features accepted	Feature completeness review
4	Validation Freeze	Evidence collection complete	Evidence sufficiency review
5	Security Freeze	No security changes after this	Security scan passage
6	Documentation Freeze	Docs complete for release	Documentation review
7	Candidate Build	Release candidate produced	Reproducible build verified
8	Verification	All gates evaluated	Quality gate passage (Vol XIII)
9	Signing	Artifacts cryptographically signed	Signature verification
10	Release	Artifacts published	Publication confirmation
11	Maintenance	Patch releases only	Support policy compliance
12	End of Life	No further releases	EOL notification period

2.2 Gate Requirements

Each lifecycle gate **MUST** be evaluated by an automated process. Gate results **MUST** be recorded in the audit framework (Volume XV) with the evaluator identity, timestamp, and result. Manual gates (such as product owner approval) **MUST** be recorded with the approver identity and a cryptographic attestation of their approval. No gate **MAY** be bypassed without a formal exception recorded in the audit trail with the exception authority, justification, and expiry. Exceptions expire after 72 hours and **MUST** be re-issued if the gate remains unmet.

2.3 Release Identifiers

Every release **MUST** have a unique identifier conforming to Semantic Versioning 2.0.0. The identifier format is MAJOR.MINOR.PATCH with optional pre-release and build metadata: MAJOR.MINOR.PATCH[-PRERELEASE][+BUILD]. Pre-release identifiers **MUST** use the format alpha.N, beta.N, rc.N where N is a positive integer. Build metadata, if present, **MUST** be the short Git SHA of the source commit. Examples: 1.0.0, 1.0.0-alpha.1, 1.0.0-rc.3+abc1234. The release identifier **MUST** appear in: (a) the Git tag, (b) the release.json artifact, (c) the provenance.json artifact, and (d) all package metadata.

Chapter 3 — Roles and Authorities

3.1 Role Definitions

The release process involves the following roles. Each role has defined authority boundaries — a role **MUST NOT** exercise authority beyond its defined scope. Multiple individuals **MAY** hold the same role. An individual **MAY** hold multiple roles provided there is no conflict of interest between the roles for any given release. The four-eyes principle **MUST** be maintained: no individual **SHALL** both produce and approve the same artifact.

Role	Responsibility	Authority	Prohibited
Developer	Write code, fix defects	Commit to non-protected branches	Approve own commits
Reviewer	Review code changes	Approve/deny pull requests	Merge without review
Release Engineer	Execute release process	Create release candidates, sign artifacts	Override quality gates
Security Officer	Security gate authority	Approve/deny security exceptions	Modify code directly
Compliance Officer	License and compliance	Approve license exceptions	Override security findings
Build Server	Automated build execution	Execute builds, publish artifacts	Modify source code
Artifact Custodian	Artifact storage and integrity	Store, retrieve, verify artifacts	Modify artifact contents

3.2 Separation of Duties

The following separation of duties **MUST** be enforced: (a) The individual who writes code for a change **MUST NOT** be the sole reviewer of that change. (b) The individual who creates a release candidate **MUST NOT** be the individual who approves the release. (c) The individual who holds the signing key **MUST NOT** be the individual who requests signing. (d) The Security Officer **MUST NOT** be the same individual as the Release Engineer for any given release. These separations are enforced through CODEOWNERS (Volume II) and protected branch policies.

1.2.1.1 Bit-Identical Reproducibility

Bit-identical reproducibility requires that two builds from the same inputs produce output artifacts that are identical at the byte level. This is the strongest form of reproducibility and is required for all VVU products where technically feasible. Bit-identical reproducibility eliminates the need to trust the build environment — if any party can reproduce the build and obtain the same bytes, the build is verified. VVU achieves bit-identical reproducibility through: deterministic compiler flags that eliminate non-determinism from compiler internals, sorted file enumeration in all packaging operations, fixed timestamps using the SOURCE_DATE_EPOCH environment variable, and hermetic build environments that eliminate host-system variance. Where bit-identical reproducibility is not feasible (e.g., due to address space layout randomization in debug builds), the product **MUST** document the variance source and **MUST** achieve functional equivalence as defined in Section 1.2.1.2.

The vres verify --reproduce command **MUST** support independent reproduction verification. This command takes a source commit, lockfile, and build configuration as inputs, executes the build in a clean hermetic environment, and compares the output artifacts against the published release artifacts. If the artifacts match at the byte level, the build is confirmed as bit-identical reproducible. If the artifacts differ, the command **MUST** produce a detailed diff report identifying which artifacts differ and the nature of the differences. The diff report **MUST** be included in the verification.json artifact for audit purposes.

1.2.1.2 Functional Equivalence

When bit-identical reproducibility is not achievable, functional equivalence provides a weaker but still valuable guarantee. Two builds are functionally equivalent if they produce artifacts that behave identically for all defined inputs, even if the binary representations differ. Functional equivalence **MUST** be verified through: (a) a test suite that achieves 100% coverage of the functional specification, (b) a cryptographic manifest that records all inputs and environmental parameters, and (c) a reproducibility statement in the release notes documenting the known variance sources. Products that claim functional equivalence rather than bit-identical reproducibility **MUST** document the technical reason why bit-identical reproducibility is not achievable and **MUST** obtain approval from the Release Engineer before release.

1.2.2.1 SOURCE_DATE_EPOCH Convention

The SOURCE_DATE_EPOCH environment variable establishes a fixed point in time for all timestamp-dependent operations during the build. When set, the build system **MUST** use this value as the current time for all operations that would otherwise use the system clock. This includes: file timestamps in archives (tar, zip), timestamps in compiled objects, timestamps in documentation generation, and timestamps in package metadata. The SOURCE_DATE_EPOCH value **MUST** be set to the commit timestamp of the source tree being built, not the current wall-clock time. This ensures that builds at different times from the same source commit produce identical timestamps in all output artifacts. The build system **MUST** verify that SOURCE_DATE_EPOCH is set before any release build and **MUST** fail if it is unset or set to an invalid value.

```
# SOURCE_DATE_EPOCH must be set before build
export SOURCE_DATE_EPOCH=$(git log -1 --format=%ct HEAD)
# Verify it is set
if [ -z "$SOURCE_DATE_EPOCH" ]; then
echo "ERROR: SOURCE_DATE_EPOCH not set"
exit 1
fi
# All build tools honor SOURCE_DATE_EPOCH
# Python: os.getenv('SOURCE_DATE_EPOCH')
# Make: override shell DATE commands
# Tar: --mtime=@$SOURCE_DATE_EPOCH
```

1.2.2.2 Sorted Enumeration

Non-determinism in file system traversal order is a common source of reproducibility failures. Different file systems and operating systems return directory entries in different orders. The build system **MUST** sort all file enumerations lexicographically before processing. This applies to: (a) file inclusion in archives (tar, zip, jar), (b) file listing for checksum computation, (c) source file compilation order, (d) resource bundling order, and (e) any operation that iterates over collections of files. The build system **MUST NOT** rely on os.listdir() or glob() results without sorting. Hash-based maps (dictionaries) **MUST** be serialized in sorted key order to ensure deterministic output regardless of hash seed randomization. Python hash randomization **MUST** be disabled in the build environment using PYTHONHASHSEED=0.

1.2.3.1 Traceability Chain

The traceability chain for a VVU release connects every distributed artifact to its provenance through a series of verifiable links. The chain begins with the source commit (Git SHA), extends through the build configuration and dependency lockfile, includes the builder identity and build environment, and terminates with the signed release artifacts. Each link in the chain **MUST** be cryptographically verified: the source commit **MUST** match the tagged release, the lockfile hash **MUST** match the recorded hash, the builder identity **MUST** be verified through Sigstore OIDC, and the artifact signatures **MUST** verify against the signing key. A break in any link invalidates the entire chain and **MUST** block the release. The traceability chain **MUST** be machine-readable (verification.json) and **MUST** be included in the enterprise procurement pack (Volume XVII) for external audit.

Traceability also extends to individual changes within a release. Every commit in the release range **MUST** be traceable to: (a) the issue or feature request it addresses, (b) the pull request that introduced it, (c) the reviewers who approved it, (d) the CI/CD pipeline run that validated it, and (e) any subsequent cherry-picks or backports. This change-level traceability enables impact analysis: given a defect, the traceability chain identifies which releases contain the defective change and which downstream artifacts are affected. The vres audit --trace command **MUST** produce this traceability chain for any specified commit or artifact.

1.2.3.2 Provenance Attestation

Provenance attestation provides cryptographic proof that a specific builder produced a specific artifact from a specific source. The attestation **MUST** include: the builder identity (OIDC email and issuer from Sigstore), the source repository URL and commit SHA, the build configuration hash, the output artifact hashes, and the build timestamp. The attestation **MUST** be signed by the builder identity through Sigstore keyless signing, which binds the signature to the OIDC identity without requiring persistent key management. Provenance attestations **MUST** be stored alongside the release artifacts as provenance.json and **MUST** be verifiable by any party using the Sigstore public good instance. This enables downstream consumers to verify that the artifact was produced by a trusted builder from the claimed source without requiring access to VVU internal systems.

1.2.4.1 Independent Verification Procedure

An independent verifier who was not involved in the release process **MUST** be able to reproduce the complete verification chain using only publicly available information. The independent verification procedure consists of the following steps: (1) Obtain the release artifacts and verification.json from the public release location. (2) Verify the checksums.txt manifest signature using the published signing key. (3) Verify each artifact hash against the manifest. (4) Verify the SLSA provenance signature and validate that the claimed source commit and build configuration are consistent. (5) Clone the source repository at the claimed commit and reproduce the build using the claimed configuration. (6) Compare the reproduced artifacts against the published artifacts. (7) Verify the SBOM against the dependency tree of the reproduced build. (8) Verify digital signatures on all signed artifacts. This procedure **MUST** be documented in the enterprise procurement pack and **MUST** be executable without access to VVU internal systems.

1.2.4.2 Audit Evidence Requirements

The audit evidence for a VVU release **MUST** include sufficient information for an external auditor to verify all claims made in the release artifacts. The minimum audit evidence set includes: (a) the complete source code at the release commit (available via the public repository), (b) the dependency lockfile with hashes, (c) the build configuration files, (d) the CI/CD pipeline execution logs (retained for 7 years), (e) the quality gate results with evaluator identities and timestamps, (f) the security scan results, (g) the compliance report with license classifications, (h) the test execution report with coverage data, and (i) the approval records with approver identities and timestamps. Audit evidence **MUST** be retained for the lifetime of the product plus seven years and **MUST** be accessible within 5 business days of an audit request. The vres audit --export command **MUST** produce a complete audit evidence bundle for any specified release.

1.2.5.1 Zero Fabrication Rule

The zero fabrication rule is absolute and non-negotiable: no VVU release artifact **MAY** contain fabricated data. Fabricated data is any value that is not derived from actual measurement, computation, or explicit user input. This includes: placeholder values that appear to be real data (e.g., a sensor reading of 0.0 that was not actually measured), interpolated values presented as measured values, and default values that mask missing data. When data is unavailable, the value **MUST** be explicitly marked using one of the approved missing-value markers: UNDEFINED (value has no meaning in this context), MISSING (value should exist but is not available), REQUIRES VALIDATION (value exists but has not been validated), or PENDING (value will be available in a future release). The vres lint --fabrication command **MUST** scan all data artifacts for potential fabrication and **MUST** report any suspected violations.

The fabrication rule has particular importance for HBK Mk-II, where sensor data informs decisions about physical water infrastructure. A fabricated pressure reading could lead to incorrect valve operations, potentially causing pipe bursts or service interruptions. For ProofBridge, a fabricated receipt amount could constitute fraud. For Ubuntu Pools, a fabricated balance could undermine the trust basis of the stokvel. These are not theoretical risks — they are concrete safety and financial risks that the zero fabrication rule addresses. Every data pipeline in a VVU product **MUST** include fabrication detection as a mandatory quality gate, and any detection **MUST** block the release.

1.2.5.2 Forbidden Terms List

The following terms are forbidden in all VVU release artifacts because they imply guarantees that have not been formally verified: SAFE_FOR_DEPLOYMENT, Engineering certified, FEA verified, Physically validated, System safe, Production ready (without qualification), Tested and verified (without specifying what was tested), and Guaranteed. These terms create false confidence and **MAY** lead to inappropriate reliance on the software in contexts where formal verification has not been performed. Instead, VVU products **MUST** use qualified language: "Verified against test suite X with Y% coverage" or "Validated per VRES quality gates G01-G19". The vres lint --forbidden-terms command **MUST** scan all release artifacts for forbidden terms and **MUST** report any occurrences as errors.

1.3.1 Conformance Levels

VRES defines three conformance levels for VVU products. Level A (Mandatory): The product satisfies all **MUST** and **SHALL** requirements defined in this standard. Level A is the minimum acceptable conformance — a product that does not conform at Level A **MUST NOT** be released. Level B (Recommended): The product satisfies all **MUST**, **SHALL**, and **SHOULD** requirements. Level B is the target conformance for all VVU products and represents full compliance with the standard. Level C (Exemplary): The product satisfies all Level B requirements and additionally implements all **MAY** provisions and exceeds minimum thresholds where specified. Level C represents best-in-class conformance and is the aspirational target for safety-critical products (HBK Mk-II, ProofBridge).

Level	Name	Requirements	Description	Applicability
Level A	Mandatory	MUST + SHALL	Minimum for release	All products
Level B	Recommended	MUST + SHALL + SHOULD	Target for all products	Standard releases
Level C	Exemplary	All requirements + MAY	Aspirational best practice	Safety-critical

1.3.2 Conformance Claims

A conformance claim **MUST** reference: (a) the specific version of this standard (VRES v1.0), (b) the conformance level achieved (A, B, or C), (c) the date of conformance assessment, (d) the assessor identity (automated system or human reviewer), and (e) any exceptions with justification. Conformance claims **MUST** be included in the release.json artifact and in the enterprise procurement pack (Volume XVII). The vres lint command **MUST** report the conformance level and **MUST** identify any requirements that are not met at the claimed level. Conformance assessment **MUST** be performed on every release — conformance claims from previous releases do **NOT** carry forward to new releases without re-assessment.

2.4 Lifecycle State Transitions

Each lifecycle state transition is governed by a mandatory gate. The following table defines the complete set of state transitions, their preconditions, postconditions, and the responsible role. A transition **MUST NOT** occur unless all preconditions are satisfied. After a transition, all postconditions **MUST** hold. The transition **MUST** be recorded in the audit trail (Volume XV) with the timestamp, the identity of the actor who authorized the transition, and the gate result that enabled the transition.

From	To	Precondition	Postcondition	Authorized By
Idea	Development	Product owner approval	Code committed to branch	Product Owner
Development	Feature Freeze	All features implemented	No new features accepted	Release Engineer
Feature Freeze	Validation Freeze	Evidence collected	Evidence immutable	Research Lead
Validation Freeze	Security Freeze	Security scan pass	No security changes	Security Officer
Security Freeze	Doc Freeze	Docs complete	Docs immutable	Tech Writer
Doc Freeze	Candidate Build	Build reproducible	RC artifacts produced	Release Engineer
Candidate Build	Verification	All gates pass	Gates recorded	Release Engineer
Verification	Signing	Verification pass	Artifacts signed	Release Engineer
Signing	Release	Sigs verified	Artifacts published	Release Engineer
Release	Maintenance	Published	Patches only	Release Engineer
Maintenance	End of Life	EOL date reached	No further releases	Product Owner

2.5 Version Policy

2.5.1 Semantic Versioning Rules

VVU products **MUST** follow Semantic Versioning 2.0.0 with the following additional constraints. MAJOR version increments indicate incompatible API changes and **MUST** be accompanied by a migration guide (MIGRATION.md) and a deprecation notice for the previous major version. MINOR version increments indicate backward-compatible functionality additions and **MUST NOT** introduce breaking changes. PATCH version increments indicate backward-compatible bug fixes and **MUST NOT** introduce new features or breaking changes. Pre-release versions (alpha, beta, rc) **MUST** be ordered according to SemVer precedence rules: alpha < beta < rc < release. Build metadata, if present, **MUST** be the 7-character abbreviated Git SHA of the source commit.

Version identifiers **MUST** appear consistently in all of the following locations: (a) the Git tag (vMAJOR.MINOR.PATCH format), (b) the release.json artifact, (c) the package metadata (package.json, pyproject.toml, Cargo.toml, go.mod as applicable), (d) the Docker image tag, (e) the documentation version footer, and (f) the API version header for versioned APIs. The vres verify --version-consistency command **MUST** check that the version identifier is identical across all these locations and **MUST** report any inconsistencies as errors.

3.3 Approval Authorities

Each lifecycle gate requires approval from a designated authority. The authority model ensures that no single individual can unilaterally advance a release through all gates. The following approval authorities are defined for each gate category. Code quality gates (G01-G07) are automatically evaluated and require no human approval — they pass or fail based on objective criteria. Security gates (G08-G09) require Security Officer review for any exception. Compliance gates (G10-G11, G18) require Compliance Officer approval. Process gates (G14, G17, G19) require the designated approver based on the gate: documentation review by Tech Writer lead, release notes by Product Owner, and final approval by Release Engineer. All approvals **MUST** be recorded with the approver identity, timestamp, and a cryptographic attestation.

3.4 Role Interaction Matrix

The following matrix defines which roles may interact with which artifacts and processes. An X indicates the role is authorized for the action. A dash indicates the role is prohibited from the action. This matrix enforces the separation of duties defined in Section 3.2 and is implemented through CODEOWNERS and protected branch policies (Volume II).

Action	Dev	Rev	RE	SO	CO	BS	AC
Write code	X	-	-	-	-	-	-
Review PR	-	X	X	-	-	-	-
Create RC	-	-	X	-	-	-	-
Sign artifacts	-	-	X	-	-	-	-
Approve security	-	-	-	X	-	-	-
Approve license	-	-	-	-	X	-	-
Execute build	-	-	-	-	-	X	-
Store artifacts	-	-	-	-	-	-	X

NOTE: RE = Release Engineer, SO = Security Officer, CO = Compliance Officer, BS = Build Server, AC = Artifact Custodian

---

Volume II

Repository Standards

Chapter 4 — Repository Structure

4.1 Monorepo Standard

VVU products **MUST** use a monorepo structure with the following top-level directories. The monorepo approach ensures that all product code, configurations, and artifacts share a single source of truth for versioning, dependency management, and audit trail generation. Products that require independent versioning **MAY** use sub-repositories, but these **MUST** be anchored to the monorepo through Git submodules with pinned commits and **MUST** be included in the monorepo SBOM. The top-level structure is as follows:

```
vvu/
src/ # Product source code
tests/ # Test suites
docs/ # Documentation sources
scripts/ # Build and release scripts
configs/ # Configuration files
prisma/ # Database schema (if applicable)
public/ # Static assets
ive-output/ # IVE evidence output
.github/ # CI/CD workflows
.zscripts/ # Integrity gate scripts
LICENSE # License file (MUST exist)
CHANGELOG.md # Changelog (structured)
RELEASE_NOTES.md # Release notes
SHA256SUMS # Checksum index
.env.example # Environment template (never .env)
```

4.2 Branch Strategy

VVU repositories **MUST** use a trunk-based development model with short-lived feature branches. The main branch (main) is the only long-lived branch and is always in a releasable state. Feature branches **MUST** be prefixed with feature/, bugfix/, hotfix/, or release/ and **MUST** be deleted after merge. Release branches (release/X.Y.Z) **MAY** be created from main for release stabilization but **MUST** be merged back or abandoned within 14 days. Protected branches (main, release/*) **MUST** require: (a) at least one approved review, (b) passing CI checks, (c) no merge conflicts, and (d) signed commits. Direct pushes to protected branches are prohibited.

4.3 Git Tagging

Release tags **MUST** follow the format vMAJOR.MINOR.PATCH (e.g., v1.0.0, v2.3.1-rc.2). Tags **MUST** be annotated (not lightweight) and **MUST** include the release identifier and a pointer to the release commit. Tags **MUST** be signed by the Release Engineer using the repository GPG key. Tag creation **MUST** be automated by the vres release command and **MUST NOT** be performed manually. Pre-release tags (alpha, beta, rc) **MUST NOT** be deleted or moved after creation — they are immutable audit records.

4.4 Commit Conventions

All commits **MUST** follow the Conventional Commits specification v1.0.0. The format is: type(scope): description. The following types are defined: feat (new feature), fix (bug fix), docs (documentation), style (formatting), refactor (code restructuring), perf (performance), test (test changes), build (build system), ci (CI/CD), chore (maintenance), revert (revert). The scope **SHOULD** reference the affected product or module. Breaking changes **MUST** include a BREAKING CHANGE footer or a ! suffix on the type. Commit messages **MUST NOT** exceed 72 characters in the first line. The commit body, if present, **SHOULD** explain the motivation for the change.

4.5 CODEOWNERS

Every repository **MUST** include a CODEOWNERS file at the root. This file defines the required reviewers for each directory and file pattern. CODEOWNERS **MUST** enforce the separation of duties defined in Chapter 3: (a) src/ **MUST** require at least one Developer review, (b) scripts/ **MUST** require Release Engineer review, (c) .github/ **MUST** require Release Engineer review, (d) LICENSE and NOTICE **MUST** require Compliance Officer review. CODEOWNERS entries **MUST** reference team names (not individual usernames) to ensure continuity when team membership changes.

4.6 Templates

Repositories **MUST** include the following templates in .github/: issue templates for bug reports, feature requests, and security vulnerabilities; pull request templates with checklists for testing, documentation, and breaking changes; and release templates that reference the VRES quality gate checklist (Volume XIII). These templates enforce consistency across all VVU product repositories and ensure that every contribution and release follows the standard process.

4.1.1 Directory Structure Details

Each top-level directory in the monorepo serves a specific purpose and has defined contents. The src/ directory contains all product source code organized by product: src/ive/, src/hbk/, src/proofbridge/, src/ubuntu-pools/, src/air-runtime/, src/trust-sphere/. Each product subdirectory **MUST** contain a README.md describing the product module structure. The tests/ directory mirrors the src/ structure with test files: tests/ive/, tests/hbk/, etc. Test files **MUST** be named test_*.py or *_test.ts following the language convention. The docs/ directory contains documentation sources in Markdown or reStructuredText format, organized by type: docs/api/ for API documentation, docs/architecture/ for architecture documents, docs/runbooks/ for operational runbooks, and docs/research/ for research documentation.

The scripts/ directory contains build, release, and operational scripts. Scripts **MUST** be executable (chmod +x) and **MUST** include a shebang line. Build scripts are in scripts/build/, release scripts in scripts/release/, and operational scripts in scripts/ops/. The configs/ directory contains configuration templates and defaults — never secrets or environment-specific values. The prisma/ directory contains the Prisma ORM schema for products using PostgreSQL. The public/ directory contains static web assets. The ive-output/ directory is the IVE evidence output directory, which **MUST** be in .gitignore for runtime output but **MUST** contain a README.md documenting the expected output structure. The .zscripts/ directory contains integrity gate scripts that are executed as part of the release verification process.

4.2.1 Branch Naming Convention

All branch names **MUST** follow the naming convention: type/JIRA-ID-short-description. The type prefix **MUST** be one of: feature/ (new functionality), bugfix/ (defect correction), hotfix/ (emergency production fix), release/ (release stabilization), refactor/ (code restructuring without behavior change), docs/ (documentation changes), chore/ (maintenance tasks), and experiment/ (experimental work that may not be merged). The JIRA-ID is the associated issue tracker identifier. The short-description is a brief hyphenated summary. Examples: feature/IVE-42-evidence-dashboard, bugfix/HBK-17-sensor-calibration, hotfix/PB-9-receipt-signing, release/1.2.0. Branch names **MUST NOT** exceed 80 characters.

4.2.2 Branch Lifecycle

Feature branches **MUST** be short-lived — the maximum lifespan is 5 business days. Branches that exceed this lifespan **MUST** be either merged or closed. Long-lived feature branches indicate insufficient decomposition and **MUST** be split into smaller branches. Release branches (release/X.Y.Z) **MAY** live up to 14 days for stabilization but **MUST** be merged back to main or abandoned after this period. All branches **MUST** be deleted after merge — the Git history preserves the branch context through the merge commit. Stale branches (no commits in 30 days) **MUST** be automatically identified and reported by the vres doctor command. The main branch **MUST** always be in a releasable state — any commit on main **MUST** pass all quality gates.

4.3.1 Tag Lifecycle and Immutability

Git tags for releases are immutable audit records. Once created, a tag **MUST NOT** be deleted or moved to a different commit. This applies to all tag types: alpha, beta, rc, and release tags. If a pre-release tag identifies a commit with a critical defect, the tag remains but the release notes **MUST** document the defect and **MUST** identify the tag as withdrawn. A new tag with an incremented pre-release number (e.g., v1.0.0-rc.4 after withdrawn v1.0.0-rc.3) **MUST** be created for the fixed commit. Tag immutability ensures that the audit trail is preserved — any tag, even one identifying a defective release, is part of the project history.

4.4.1 Commit Message Examples

The following examples illustrate the required commit message format for common scenarios. Each message follows the Conventional Commits specification with a type, optional scope, description, and optional body and footer.

```
feat(ive): add evidence dashboard panel 14
Implement the calibration status panel with real-time
sensor data and Brier Score visualization.

Refs: IVE-42

fix(hbk): correct pressure sensor offset calculation
The pressure offset was applied after unit conversion
instead of before, causing a 2.3% systematic error.

Fixes: HBK-17
BREAKING CHANGE: sensor readings will change for existing deployments

docs(deployment): add air-gapped installation procedure

chore(deps): update numpy to 1.26.4 for CVE-2024-XXXX

refactor(proofbridge): extract receipt signing into safety kernel
```

4.5.1 CODEOWNERS Example

The CODEOWNERS file enforces required reviewers for each area of the codebase. The following example demonstrates the required structure for a VVU monorepo.

```
# CODEOWNERS — VVU Monorepo
# Enforces separation of duties per VRES Volume I, Chapter 3

# Default — all changes require at least one developer review
* @vvu/developers

# Source code — requires product team review
/src/ive/ @vvu/ive-team
/src/hbk/ @vvu/hbk-team
/src/proofbridge/ @vvu/pb-team
```

```
# Build and release — requires Release Engineer review
/scripts/ @vvu/release-engineers
/.github/ @vvu/release-engineers
/configs/ @vvu/release-engineers

# Security — requires Security Officer review
/SECURITY.md @vvu/security-officers
/scripts/security/ @vvu/security-officers

# License and compliance — requires Compliance Officer
/LICENSE @vvu/compliance-officers
/NOTICE @vvu/compliance-officers
/COMMERCIAL-LICENSE.md @vvu/compliance-officers

# Documentation — requires tech writing review
/docs/ @vvu/tech-writers
```

4.6.1 Pull Request Template

Every pull request **MUST** use the PR template defined in .github/pull_request_template.md. The template ensures that all required information is provided for review and that the reviewer can verify compliance with VRES requirements.

```
## Description


## Type of Change
- [ ] feat: New feature
- [ ] fix: Bug fix
- [ ] docs: Documentation
- [ ] refactor: Code restructuring
- [ ] perf: Performance improvement
- [ ] test: Test changes
- [ ] chore: Maintenance

## Checklist
- [ ] All tests pass locally
- [ ] New tests added for new functionality
- [ ] Documentation updated (if applicable)
- [ ] CHANGELOG.md updated
- [ ] No secrets or credentials included
- [ ] Conventional Commits format used
- [ ] Breaking changes documented (if applicable)

## Quality Gates
- [ ] G01: Formatting passes
- [ ] G02: Lint passes
- [ ] G03: Type safety passes
- [ ] G04: Static analysis passes
- [ ] G05: Unit tests pass
- [ ] G08: No critical/high CVEs
- [ ] G09: No secrets detected
```

4.6.2 Release Template

Release templates define the checklist and process for creating a release. The template **MUST** be located at .github/ISSUE_TEMPLATE/release.md and **MUST** include all VRES quality gates (Volume XIII) as checklist items. The release template **MUST** reference the specific VRES version and **MUST** include fields for: the release version identifier, the release type (major, minor, patch, hotfix), the source branch, the target deployment topologies, and any known issues or breaking changes. The release template **MUST** be approved by the Release Engineer and **MUST NOT** be modified without going through the standard review process.

Volume III

Build Engineering

Chapter 5 — Build System Requirements

5.1 Supported Languages

VVU products span multiple programming languages, each with specific build requirements. The build system **MUST** support deterministic compilation for all languages in the product. Where a language or toolchain does not natively support deterministic output, the build system **MUST** implement compensating controls (e.g., sorting, fixed timestamps, reproducible archive ordering) to achieve reproducibility as defined in Section 1.2.1. The following languages are supported with the specified build tools and reproducibility mechanisms:

Language	Build Tool	Lock Mechanism	Reproducibility Strategy
Python	uv / pip	pyproject.toml + uv.lock	Lockfile + hash pinning
Go	go build	go.mod + go.sum	Module checksum database
Rust	cargo	Cargo.lock	Cargo.lock + vendoring
TypeScript	bun / tsc	package.json + bun.lock	Lockfile + integrity hashes
C++	cmake + ninja	CMakeLists.txt + lock	SOURCE_DATE_EPOCH + sorted arcs
CUDA	nvcc + cmake	Same as C++	Deterministic compiler flags
ROCm	hipcc + cmake	Same as C++	Deterministic compiler flags

5.2 Container Builds

Container images **MUST** be built using multi-stage Dockerfiles that separate the build environment from the runtime environment. The runtime stage **MUST** use a minimal base image (distroless or alpine) and **MUST NOT** include build tools, source code, or debug symbols. Container images **MUST** be built with BuildKit and **MUST** use the --no-cache flag to ensure reproducibility. Image tags **MUST** use the release version identifier (not latest). Every container image **MUST** include: (a) a label with the source repository URL, (b) a label with the source commit SHA, (c) a label with the SBOM reference, and (d) a HEALTHCHECK instruction where applicable. Container images **MUST** be scanned for vulnerabilities (Volume V) before publication.

5.3 Cross-Platform Builds

Products that target multiple platforms **MUST** produce builds for all supported platforms from a single source commit. The supported platform matrix is: linux/amd64, linux/arm64, windows/amd64 (if applicable), and macOS/arm64 (if applicable). Cross-platform builds **MUST** be verified through platform-specific test suites (Volume VI) and **MUST** produce identical functional behavior across platforms. Platform-specific compilation flags **MUST** be documented in the build configuration and recorded in the audit trail.

5.4 Hermetic Builds

Build environments **MUST** be hermetic: they **MUST NOT** depend on tools, libraries, or configuration from the host system. All build dependencies **MUST** be explicitly declared and fetched from a controlled artifact repository. Network access during builds **MUST** be restricted to declared artifact repositories only — no general internet access is permitted. The build system **MUST** verify that all fetched dependencies match their declared hashes before use. Hermetic builds enable offline builds for air-gapped environments (Volume XI) and eliminate the class of build failures caused by environmental drift.

5.5 Offline and Air-Gapped Builds

Products targeted for air-gapped deployment (government, research clusters) **MUST** support fully offline builds. This requires: (a) a vendored dependency directory containing all required packages and their transitive dependencies, (b) a local mirror of all required container base images, (c) build scripts that detect offline mode and use vendored resources, and (d) a vres doctor --offline command that verifies all dependencies are available locally. The vendored dependency set **MUST** be regenerated on every dependency update and **MUST** be included in the release artifacts for air-gapped deployment scenarios.

5.1.1 Python Build Configuration

Python products **MUST** use uv as the primary package manager with pyproject.toml as the build configuration. The pyproject.toml **MUST** specify: the project name, version (dynamic from git), required Python version (>= 3.12), dependencies with exact version pins in the lockfile, and build system configuration. The uv.lock lockfile **MUST** be committed to the repository and **MUST** be regenerated on every dependency change. Hash verification **MUST** be enabled for all dependencies to ensure integrity. The build **MUST** produce a wheel (.whl) and source distribution (.tar.gz) with deterministic metadata.

```
[project]
name = "ive"
version = "0.0.0" # Dynamic from git tag
requires-python = ">=3.12"
dependencies = [
"fastapi>=0.115.0,<0.116.0",
"pydantic>=2.10.0,<2.11.0",
"numpy>=1.26.0,<1.27.0",
]

[build-system]
requires = ["hatchling", "hatch-vcs"]
build-backend = "hatchling.build"

[tool.hatch.version]
source = "vcs"

[tool.ruff]
line-length = 100
target-version = "py312"
```

```
[tool.mypy]
python_version = "3.12"
strict = true
```

5.1.2 Rust Build Configuration

Rust products (ProofBridge safety kernel, HBK Mk-II computation engine) **MUST** use Cargo with Cargo.lock committed to the repository. The Cargo.toml **MUST** specify: the crate name, version (matching the VRES release version), edition (2021), and all dependencies with exact version pins. The Cargo.lock file ensures deterministic builds across environments. For release builds, the profile **MUST** enable LTO (link-time optimization), strip debug symbols, and set opt-level = 3. The build **MUST** produce a static binary where possible to eliminate runtime dependency on shared libraries. Cross-compilation for ARM64 targets **MUST** be supported through cargo build --target aarch64-unknown-linux-gnu.

```
[package]
name = "proofbridge-kernel"
version = "1.0.0"
edition = "2021"

[dependencies]
ed25519-dalek = { version = "2.1.1", features = ["serde"] }
merklemountainrange = "0.3.0"
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"

[profile.release]
lto = true
strip = true
opt-level = 3
codegen-units = 1 # Deterministic codegen order
```

5.1.3 TypeScript Build Configuration

TypeScript products (IVE dashboard, Ubuntu Pools frontend) **MUST** use bun as the runtime and package manager. The package.json **MUST** specify exact version pins for all dependencies and the bun.lockb lockfile **MUST** be committed. TypeScript compilation **MUST** use strict mode with no implicit any, no unused variables, and no fallthrough in switch statements. The build **MUST** produce both ESM and CJS outputs for library packages, and a bundled output for application packages. Source maps **MUST** be generated for debug builds but **MUST NOT** be included in release artifacts. Tree-shaking **MUST** be enabled for production builds to minimize bundle size.

5.2.1 Dockerfile Template

All container images **MUST** use multi-stage builds. The following template demonstrates the required structure for a VVU Python application container.

```
# Stage 1: Build
FROM python:3.12-slim AS builder
WORKDIR /build
COPY pyproject.toml uv.lock ./
RUN pip install --no-cache-dir uv && uv sync --frozen --no-dev
COPY . .
RUN uv build

# Stage 2: Runtime
FROM python:3.12-slim AS runtime
WORKDIR /app
# Run as non-root
```

```
RUN addgroup --system app && adduser --system --ingroup app app
COPY --from=builder /build/dist /app/dist
RUN pip install --no-cache-dir /app/dist/*.whl
USER app
# Labels for traceability
LABEL org.vvu.source="https://github.com/vvu/ive"
LABEL org.vvu.commit="%%COMMIT_SHA%%"
LABEL org.vvu.sbom="%%SBOM_HASH%%"
HEALTHCHECK --interval=30s --timeout=5s \
CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
EXPOSE 8000
CMD ["python", "-m", "ive"]
```

5.3.1 Cross-Platform Build Matrix

The cross-platform build matrix defines the platforms and architectures that each VVU product **MUST** support. The matrix is evaluated in CI/CD (Volume XIV) on every release candidate.

Product	linux/amd64	linux/arm64	win/amd64	macOS/x64	macOS/arm64	Container
IVE	X	X	-	-	X	X
HBK Mk-II	X	X	-	-	X	-
ProofBridge	X	X	-	-	X	-
Ubuntu Pools	X	X	-	-	X	X
AIR Runtime	X	X	-	-	X	X
Trust Sphere	X	X	-	-	-	-

5.4.1 Hermetic Build Environment

The hermetic build environment is implemented using Nix or Docker with the following constraints: (a) The build environment is defined entirely by a lockfile — no implicit dependencies on the host system. (b) Network access is restricted to declared artifact repositories (PyPI, crates.io, npmjs.com) with hash verification. (c) All tools and compilers are pinned to exact versions in the build environment specification. (d) The build environment is reproducible — given the same lockfile, the same environment is produced on any host. (e) Environment variables that could affect build output (PATH, LD_LIBRARY_PATH, locale) are fixed in the build specification. The vres doctor --hermetic command **MUST** verify that the build environment is hermetic by checking for undeclared dependencies and network access beyond the declared repositories.

5.5.1 Offline Build Procedure

The offline build procedure enables VVU products to be built in air-gapped environments commonly found in South African government and research institutions. The procedure is: (1) On an internet-connected machine, run vres vendor to download all dependencies into the vendor/ directory. (2) Transfer the complete source tree (including vendor/) to the air-gapped environment via physical media or approved transfer protocol. (3) On the air-gapped machine, run vres doctor --offline to verify all dependencies are available. (4) Run the build with --offline flag, which configures all package managers to use the vendor/ directory instead of network repositories. (5) Verify the build output matches the expected hashes from the online build. The vendor/ directory **MUST** be regenerated on every dependency update and **MUST** be included in the release artifacts for air-gapped deployment scenarios (Volume XI, Section 13.2).

```
# Step 1: Vendor dependencies (online machine)
vres vendor --output vendor/
# Creates:
# vendor/python/ — Python wheels
# vendor/rust/ — Rust crate sources
# vendor/npm/ — NPM packages
# vendor/containers/ — Base container images (tar)

# Step 3: Verify offline readiness
vres doctor --offline

# Step 4: Build offline
vres build --offline --vendor-dir=vendor/

# Step 5: Verify output
vres verify --manifest=checksums.txt
```

Volume IV

Dependency Governance

Chapter 6 — Dependency Management

6.1 Software Bill of Materials (SBOM)

Every release **MUST** include an SBOM in both SPDX and CycloneDX formats. The SBOM **MUST** enumerate all direct and transitive dependencies, their versions, their licenses, and their supplier (origin) information. The SBOM **MUST** be generated automatically during the build process using a tool that extracts dependency information directly from the package manager lockfiles — manual SBOM construction is prohibited. The SBOM **MUST** be validated against the actual dependency tree to ensure completeness and accuracy. The SBOM **MUST** be included in the release artifacts as both sbom.spdx.json and sbom.cyclonedx.json.

6.2 Dependency Pinning

All dependencies **MUST** be pinned to exact versions in lockfiles. Range specifiers (e.g., ^1.2.0, >=2.0.0) are prohibited in production lockfiles. Lockfiles **MUST** be committed to the repository and **MUST** be regenerated whenever dependencies are added, removed, or updated. The vres sbom command **MUST** verify that the lockfile matches the installed dependency tree and **MUST** report any discrepancies as errors. Dependency updates **MUST** go through the standard pull request process with automated vulnerability scanning (Section 6.4).

6.3 License Compatibility Matrix

Every dependency **MUST** have its license classified according to the following compatibility matrix. VVU products use a dual-license model (Section 24.1): AGPL-3.0-or-later for the open-source track and a commercial license for the enterprise track. Dependencies with incompatible licenses **MUST NOT** be included in any VVU product. The Compliance Officer **MUST** approve any exception.

License	AGPL-3.0 Track	Commercial Track	Notes
Apache-2.0	Compatible	Compatible	Permissive, strong patent grant
MIT	Compatible	Compatible	Permissive, minimal restrictions

License	AGPL-3.0 Track	Commercial Track	Notes
BSD-2-Clause	Compatible	Compatible	Permissive, two-clause
BSD-3-Clause	Compatible	Compatible	Permissive, three-clause
ISC	Compatible	Compatible	Permissive, functionally equivalent to MIT/BSD
GPL-2.0-only	Incompatible	Conditional	Copyleft, requires careful analysis
GPL-3.0-only	Compatible	Conditional	Copyleft, compatible with AGPL-3.0
AGPL-3.0-only	Compatible	Incompatible	Strong copyleft, network clause
LGPL-2.1-only	Conditional	Conditional	Weak copyleft, linking exception
MPL-2.0	Conditional	Conditional	File-level copyleft
Unlicense	Compatible	Compatible	Public domain dedication
Proprietary	Incompatible	Conditional	Requires commercial license agreement

6.4 Vulnerability Scanning

Dependencies **MUST** be scanned for known vulnerabilities on every pull request that modifies lockfiles and on a scheduled basis (at minimum daily) for the main branch. The scanning tool **MUST** consult the National Vulnerability Database (NVD), GitHub Advisory Database, and OSV database. Critical and high severity vulnerabilities **MUST** block the release pipeline until remediated. Medium severity vulnerabilities **SHOULD** be remediated before release but **MAY** be accepted with documented justification and an exception from the Security Officer. Low severity vulnerabilities **SHOULD** be tracked for future remediation. The vres lint command **MUST** report the count and severity of known vulnerabilities in the dependency tree.

6.5 Supply-Chain Policy

VVU adheres to the SLSA (Supply-chain Levels for Software Artifacts) framework. All VVU products **MUST** achieve SLSA Level 3 for production releases, which requires: (a) hermetic builds with no network access, (b) provenance generation documenting the build process, (c) non-falsifiable provenance signed by the build platform, and (d) verified provenance before deployment. SLSA Level 4 (two-party review of all changes) is the target for safety-critical products (HBK Mk-II Tier 1 firmware). The supply-chain policy **MUST** be documented in the repository SECURITY.md file and **MUST** reference the SLSA level achieved.

6.1.1 SPDX SBOM Format

The SPDX (System Package Data Exchange) SBOM format is the primary SBOM format for VVU products. The SPDX document **MUST** conform to SPDX Specification 2.3 and **MUST** include: a DocumentNamespace that uniquely identifies the SBOM, a creationInfo section with the tool name and version used to generate the SBOM, a package listing for each direct and transitive dependency, relationship entries linking packages to their dependencies, and license conclusions for each package using SPDX License Identifiers. The SPDX SBOM **MUST** be generated automatically from the package manager lockfile — manual SBOM creation is prohibited. The vres sbom --format spdx command generates the SPDX SBOM.

```
{
"spdxVersion": "SPDX-2.3",
"dataLicense": "CC0-1.0",
"SPDXID": "SPDXRef-DOCUMENT",
"documentNamespace": "https://vvu.org/spdx/ive-1.2.3",
"name": "IVE",
"creationInfo": {
```

```
"created": "2026-08-06T12:00:00Z",
"creators": ["Tool: vres-sbom-1.0.0"]
},
"packages": [
{
"SPDXID": "SPDXRef-Package-fastapi",
"name": "fastapi",
"versionInfo": "0.115.0",
"licenseConcluded": "MIT",
"supplier": "Organization: Sebastián Ramírez"
}
]
}
```

6.1.2 CycloneDX SBOM Format

The CycloneDX SBOM format is the secondary format, included for compatibility with enterprise security tools that prefer CycloneDX. The CycloneDX document **MUST** conform to CycloneDX Specification 1.5 and **MUST** include the same information as the SPDX SBOM but in CycloneDX format: component entries with name, version, type, and license, dependency graph relationships, and vulnerability data if available. Both SPDX and CycloneDX SBOMs **MUST** be generated from the same data source to ensure consistency. The vres sbom command generates both formats simultaneously. The vres verify --sbom command **MUST** validate that both formats contain the same dependency set and **MUST** report any discrepancies.

6.2.1 Pinning Examples per Language

Dependency pinning **MUST** use exact version specifiers in production lockfiles. Range specifiers are prohibited. The following examples show the required pinning format for each supported language. Python (pyproject.toml with uv.lock): dependencies in pyproject.toml **MAY** use compatible ranges (>=, ~=) but uv.lock **MUST** contain exact versions with hashes. Rust (Cargo.lock): Cargo.toml **MAY** use SemVer ranges but Cargo.lock **MUST** be committed with exact versions. TypeScript (package.json with bun.lockb): package.json **MAY** use ranges but bun.lockb **MUST** be committed. Go (go.mod with go.sum): go.mod **MAY** use ranges but go.sum **MUST** contain integrity hashes. C++ (CMakeLists.txt with Conan lockfile): CMakeLists.txt specifies versions but the Conan lockfile **MUST** pin exact versions with hashes.

```
# Python: uv.lock (exact pins with hashes)
fastapi == 0.115.0 --hash=sha256:abc123...
pydantic == 2.10.0 --hash=sha256:def456...

# Rust: Cargo.lock (exact pins)
[[package]]
name = "ed25519-dalek"
version = "2.1.1"
source = "registry+https://github.com/rust-lang/crates.io-index"

# Go: go.sum (integrity hashes)
github.com/go-chi/chi v1.5.5 h1:abc123...

# TypeScript: bun.lockb (binary lockfile with integrity)
```

6.4.1 Vulnerability Severity Handling

Vulnerability severity determines the required response time and the impact on the release pipeline. The following severity handling matrix defines the required action for each level.

Severity	Pipeline Impact	Fix Deadline	Required Action	Authority
Critical (CVSS >= 9.0)	Block release	24 hours	MUST fix or document exception	Security Officer
High (CVSS >= 7.0)	Block release	72 hours	MUST fix or document exception	Security Officer
Medium (CVSS >= 4.0)	Blockable	14 days	SHOULD fix, MAY defer with justification	Release Engineer
Low (CVSS < 4.0)	Advisory	Next release	Track for future remediation	Developer
Informational	No action	No deadline	Document in security report	Developer

6.5.1 SLSA Level Requirements per Product

Each VVU product has a minimum SLSA level requirement based on its criticality and deployment context. Products operating in safety-critical contexts (HBK Mk-II in water infrastructure) target the highest SLSA level. Products with financial integrity requirements (ProofBridge) also target higher levels. The following table defines the minimum and target SLSA levels.

Product	Minimum	Target	Rationale
IVE	SLSA 3	SLSA 3	Enterprise verification platform
HBK Mk-II	SLSA 3	SLSA 4	Safety-critical water infrastructure
ProofBridge	SLSA 3	SLSA 4	Financial receipt integrity
Ubuntu Pools	SLSA 3	SLSA 3	Community savings platform
AIR Runtime	SLSA 3	SLSA 3	Application runtime
Trust Sphere	SLSA 3	SLSA 4	Audit and trust infrastructure

Volume V

Security Engineering

Chapter 7 — Security Requirements

7.1 Secret Detection

Repositories **MUST** be scanned for accidental credential disclosure on every commit. The scanning tool **MUST** detect: private keys (RSA, ECDSA, Ed25519), API keys and tokens, database connection strings, passwords in configuration files, and cloud credentials (AWS, GCP, Azure). Detected secrets **MUST** block the commit or pull request and **MUST** be reported to the Security Officer. Committed secrets **MUST** be rotated immediately regardless of whether they were used in production. The .gitignore file **MUST** exclude .env files, credential files, and key directories. Pre-commit hooks for secret scanning **MUST** be enabled in all VVU repositories.

7.2 Static and Dynamic Analysis

Static Application Security Testing (SAST) **MUST** be performed on every pull request. The SAST tool **MUST** cover: injection vulnerabilities (SQL, command, LDAP), cross-site scripting (reflected and stored), insecure deserialization, path traversal, insecure defaults, and cryptographic misuse. SAST findings of critical or high severity **MUST** block the pull request. Dynamic Application Security Testing (DAST) **MUST** be performed on release candidates against a deployed test environment. DAST **MUST** cover the OWASP Top 10 and **MUST** be configured with product-specific authentication. Both SAST and DAST results **MUST** be included in the release artifacts as security-report.json.

7.3 Artifact Signing

All release artifacts **MUST** be cryptographically signed. The signing system **MUST** use Ed25519 keys managed through Sigstore/Cosign for keyless signing with OIDC identity-based certificates. For products requiring traditional key management (HBK Mk-II firmware, ProofBridge receipts), signing **MUST** use Ed25519 keys stored in an HSM with threshold custody (3-of-5 Shamir Secret Sharing as implemented in SafeKrypte). Every signed artifact **MUST** include: (a) the signature file (artifact.sig), (b) the public key or certificate used for verification, and (c) the signing timestamp from a trusted time source. Key rotation **MUST** be performed annually and **MUST** not invalidate previously signed artifacts — old keys **MUST** be retained for verification.

7.4 Container Security

Container images **MUST** be scanned for vulnerabilities before publication using a container-specific scanner that checks: (a) the base image for known CVEs, (b) installed packages for known vulnerabilities, (c) configuration for security best practices (running as non-root, read-only filesystem, no privilege escalation), and (d) embedded secrets or credentials. Critical vulnerabilities **MUST** block publication. Containers **MUST** run as non-root users with dropped capabilities. The container security scan results **MUST** be included in the release artifacts and in the enterprise procurement pack (Volume XVII).

7.5 Key Custody and Rotation

Signing keys **MUST** be managed according to the following custody model: (a) Development signing keys are managed through Sigstore keyless signing — no persistent keys required. (b) Release signing keys are stored in an HSM with threshold custody (k-of-n Shamir Secret Sharing). (c) Key custodians **MUST** be named individuals with documented identity verification. (d) Key material **MUST NOT** be stored in repositories, CI/CD configurations, or environment variables. (e) Key rotation **MUST** be performed on schedule (annually for release keys, quarterly for deployment keys) and on event (key custodian departure, suspected compromise). (f) Retired keys **MUST** be archived with their signing history for the lifetime of any artifact they signed, to enable signature verification of historical releases.

7.6 Incident Response

Security incidents affecting released artifacts **MUST** follow the incident response procedure: (a) Detection: Automated monitoring and vulnerability scanning (Section 6.4) **MUST** detect known vulnerabilities within 24 hours of CVE publication. (b) Assessment: The Security Officer **MUST** assess severity and impact within 4 hours of detection. (c) Containment: If the vulnerability affects released artifacts, the release **MUST** be marked as vulnerable in the artifact repository and affected versions **MUST** be listed in the security advisory. (d) Remediation: A fix **MUST** be developed and released following the hotfix process (Volume XII, Section 12.7). (e) Communication: Security advisories **MUST** be published within 72 hours of remediation and **MUST** include: the CVE identifier, affected versions, severity, and remediation instructions.

7.1.1 Secret Detection Configuration

The secret detection tool **MUST** be configured with rules for all known credential types relevant to VVU products. The configuration **MUST** specify: (a) patterns for private key formats (PEM, OpenSSH, PuTTY), (b) patterns for API key formats (AWS access keys, GitHub tokens, Stripe keys, Stitch API keys), (c) patterns for database connection strings (PostgreSQL, SQLite with paths), (d) patterns for South African specific credentials (SARS eFiling, municipal API keys, bank account numbers), and (e) allowlisted patterns for test fixtures and documentation examples. The allowlist **MUST** be reviewed quarterly by the Security Officer. The detection tool **MUST** run as a pre-commit hook and as a CI/CD gate.

7.2.1 SAST Tool Configuration

SAST tools **MUST** be configured per language in the product. Python products **MUST** use Bandit with the VRES rule configuration that enables all checks and sets severity thresholds: critical and high findings block the pipeline, medium findings generate warnings. TypeScript products **MUST** use ESLint with the security plugin (eslint-plugin-security) and the no-eval, no-implied-eval, and no-new-func rules enabled. Rust products **MUST** use cargo-audit for dependency vulnerability scanning and clippy with the nursery lint group for code quality. All SAST results **MUST** be in SARIF format for interop with GitHub Code Scanning and **MUST** be included in the security-report.json artifact.

7.3.1 Signing Procedure

The artifact signing procedure follows these steps: (1) The build completes and produces all unsigned artifacts. (2) The checksums.txt manifest is generated from the unsigned artifacts. (3) The Release Engineer invokes vres sign, which: (a) generates an Ed25519 signature of checksums.txt using Sigstore/Cosign keyless signing for open-source products or HSM-backed signing for enterprise products, (b) generates individual signatures for critical artifacts (firmware, receipts, evidence packages), and (c) records the signing event in the audit trail. (4) The vres verify command is run to verify all signatures. (5) If all signatures verify, the release proceeds to the Release state. Signing keys **MUST NOT** be accessible to developers or CI/CD pipelines — signing is a controlled operation performed by the Release Engineer or an automated signing service with threshold approval.

```
# Signing procedure using Sigstore/Cosign
# Step 1: Generate manifest
vres manifest --output checksums.txt

# Step 2: Sign manifest (keyless via OIDC)
cosign sign-blob checksums.txt \
--yes \
--certificate-identity=release-engineer@vvu.org \
--certificate-oidc-issuer=https://accounts.google.com

# Step 3: Sign container image
cosign sign registry.vvu.org/ive:1.2.3 \
--yes

# Step 4: Verify signatures
cosign verify-blob checksums.txt \
--signature checksums.sig \
--certificate checksums.crt \
--certificate-identity=release-engineer@vvu.org \
--certificate-oidc-issuer=https://accounts.google.com
```

7.5.1 Key Management Lifecycle

Signing keys follow a defined lifecycle from creation through retirement. Key creation: new Ed25519 key pairs are generated in the HSM with threshold custody (3-of-5 Shamir Secret Sharing). Key activation: the key is registered in the Sigstore transparency log and its public key is published in the VVU key store. Key rotation: performed annually for release keys and quarterly for deployment keys, or on event (custodian departure, suspected compromise). Key retirement: the key is marked as retired but its public key is retained for verification of historical signatures. Key destruction: key material is destroyed only after all artifacts signed with the key have been re-signed with the current key or have exceeded their retention period. The key lifecycle **MUST** be documented in the security audit trail and the current key status **MUST** be available via vres audit --keys.

7.6.1 Incident Severity Matrix

The incident severity matrix defines the classification, response requirements, and escalation paths for security incidents affecting VVU products.

Severity	Description	Response	Remediation	Escalation
SEV1 - Critical	Active exploitation, data breach, safety impact	15 min	1 hour	CTO, Security Officer, Legal
SEV2 - High	Confirmed vulnerability in released artifact	1 hour	24 hours	Security Officer, Release Engineer
SEV3 - Medium	Vulnerability in dependency, not yet exploited	4 hours	7 days	Security Officer
SEV4 - Low	Informational finding, best practice gap	1 day	30 days	Developer

7.4.1 Container Security Checklist

Every container image **MUST** pass the following security checklist before publication. The checklist is evaluated by the container scanner and by vres verify --container.

Check	Description	Req.	Verification
C01	Base image has no critical CVEs	MUST	Trivy/Grype scan
C02	Runs as non-root user	MUST	Dockerfile USER directive
C03	No privilege escalation	MUST	no-new-privileges:true
C04	Read-only filesystem where possible	MUST	readOnlyRootFilesystem: true
C05	No secrets in image layers	MUST	Secret scanning in build context
C06	Minimal base image (distroless/alpine)	MUST	Image size audit
C07	Health check defined	MUST	HEALTHCHECK instruction

Check	Description	Req.	Verification
C08	Resource limits defined	SHOULD	Kubernetes resource limits
C09	Network policies defined	SHOULD	Kubernetes NetworkPolicy
C10	Image signed with Cosign	MUST	Cosign verify

Volume VI

Testing

Chapter 8 — Test Requirements

8.1 Test Categories

VVU products **MUST** implement the following test categories. Each category has specific coverage requirements and gate thresholds. Tests **MUST** be automated, deterministic, and hermetic — they **MUST NOT** depend on external services, shared state, or non-deterministic inputs. Test results **MUST** be recorded in a machine-readable format (test-report.json) and included in the release artifacts. All test categories **MUST** pass for a release to proceed past the Verification state (Chapter 2, State 8).

Category	Scope	Threshold	Frequency
Unit	Individual functions and classes	>= 80% line coverage	Every PR
Integration	Component interactions	All critical paths	Every PR
End-to-End	Full system workflows	All user stories	Release candidate
Regression	Known defect prevention	All fixed defects	Every PR
Stress	Resource limits and saturation	No degradation below SLO	Release candidate
Load	Expected and peak traffic	Response time within budget	Release candidate
Performance	Benchmark against budgets	No regression > 5%	Release candidate
Reliability	Failure recovery and resilience	MTTR within SLO	Release candidate
Fault Injection	Error path coverage	Graceful degradation	Release candidate
Chaos Engineering	Infrastructure resilience	Recovery without data loss	Monthly

8.2 Hardware-in-the-Loop Testing

Products with hardware dependencies (HBK Mk-II, firmware components) **MUST** include hardware-in-the-loop (HIL) tests. HIL tests validate software behavior against real or emulated hardware interfaces. For HBK Mk-II, HIL tests **MUST** cover: (a) pressure sensor input ranges and calibration verification, (b) actuator command verification, (c) communication bus integrity (CAN, Modbus), and (d) power supply behavior under brownout and surge conditions. HIL test results are mandatory for HBK Mk-II releases and **MUST** be included in the evidence package (Volume XVI). Where physical hardware is unavailable, a validated hardware emulation **MUST** be used, with the emulation fidelity documented in the test report.

8.3 Product-Specific Validation Tests

Each VVU product **MUST** define product-specific validation tests that verify the product satisfies its domain requirements. These tests go beyond functional correctness to verify domain integrity: HBK Mk-II: Brier Score below TRIP threshold (0.02), MCMC convergence diagnostics, hydraulic simulation conservation of mass/energy within tolerance. ProofBridge: receipt integrity (Ed25519 signature verification), MMR append-only property, ZK-proof generation correctness. IVE: evidence package completeness, frozen contract schema validation, adapter attribution integrity (no fabricated fields), pipeline preservation (historical runs unchanged). Ubuntu Pools: stokvel balance integrity (sum of contributions equals vault total), ProofBridge receipt for every contribution, Stitch payment verification.

8.1.1 Unit Test Configuration

Unit tests **MUST** be configured with the following requirements: Python products use pytest with pytest-cov for coverage reporting, configured in pyproject.toml with strict markers and xfail strict mode. TypeScript products use Vitest with coverage-v8 provider. Rust products use cargo test with nextest for parallel execution. All unit tests **MUST** be deterministic — no test **MAY** depend on the current time, random values (without seeded RNG), network access, or file system state outside the test fixture. Test isolation is enforced through test fixtures that set up and tear down the test environment for each test case. Tests that violate isolation (e.g., by writing to shared state) **MUST** be identified and fixed before merge.

```
# pytest configuration in pyproject.toml
[tool.pytest.ini_options]
minversion = "8.0"
addopts = [
"--strict-markers",
"--strict-config",
"--cov=ive",
"--cov-fail-under=80",
"--cov-report=term-missing",
"--cov-report=json",
]
markers = [
"unit: Unit tests",
"integration: Integration tests",
"e2e: End-to-end tests",
"hil: Hardware-in-the-loop tests",
"slow: Slow tests (deselect with -m 'not slow')",
]
xfail_strict = true
```

8.1.2 Coverage Thresholds per Product

Each VVU product has specific coverage thresholds based on its criticality. Safety-critical products have higher thresholds because defects in these products could cause physical harm or financial loss.

Product	Unit Coverage	Integration	E2E	Branch
IVE	>= 80%	>= 70%	>= 60%	>= 50%
HBK Mk-II	>= 90%	>= 85%	>= 80%	>= 70%
ProofBridge	>= 90%	>= 85%	>= 75%	>= 60%
Ubuntu Pools	>= 80%	>= 70%	>= 60%	>= 50%
AIR Runtime	>= 80%	>= 70%	>= 60%	>= 50%
Trust Sphere	>= 85%	>= 80%	>= 70%	>= 60%

8.2.1 HIL Test Procedure

The Hardware-in-the-Loop (HIL) test procedure for HBK Mk-II is as follows: (1) The test environment is initialized with the hardware emulator or physical hardware connected to the test runner. (2) Sensor calibration data is loaded from the calibration certificates on file. (3) The test suite executes against the release candidate firmware, sending simulated sensor inputs and verifying actuator commands. (4) Communication bus integrity is verified (CAN bus error rates, Modbus response times). (5) Power supply behavior is tested under brownout (voltage sag to 80% nominal) and surge (150% nominal for 100ms) conditions. (6) The HIL test report is generated with pass/fail results, timing data, and hardware identification. HIL tests **MUST** execute in under 2 hours for release candidate verification. HIL test results **MUST** be included in the evidence package (Volume XVI) and in the HBK Mk-II product overlay (Volume XIX, Section 21.3).

8.1.3 Chaos Engineering Scenarios

Chaos engineering tests validate the resilience of VVU deployments under adverse conditions. The following scenarios **MUST** be executed monthly for production deployments and **MUST** be available as automated tests for release candidates.

Scenario	Failure Mode	Expected Outcome	Applicable
Pod Kill	Random pod termination	Service recovers within RTO	Kubernetes deployments
Network Partition	Network split between zones	Degraded service, no data loss	HA topology
Disk Fill	Disk usage to 95%	Alerting triggers, graceful degradation	All topologies
CPU Stress	CPU at 100% for 5 minutes	Response time within 2x SLO	All topologies
Dependency Failure	External service unavailable	Circuit breaker activates, fallback works	Cloud topology
DB Connection Pool Exhaust	All DB connections consumed	Connection queue, timeout handling	All topologies
Memory Leak Simulation	Gradual memory increase	OOM prevention, restart recovery	All topologies

8.3.1 Test Reporting Format

All test results **MUST** be reported in a standardized JSON format (test-report.json) that enables automated quality gate evaluation and audit trail recording. The format **MUST** include: the test execution timestamp, the test environment identification, each test case with its result (PASS/FAIL/SKIP), duration, and failure details, the coverage data with line, branch, and function coverage percentages, and a summary with total, passed, failed, and skipped counts. The test-report.json **MUST** be a release artifact (Volume IX) and **MUST** be included in the enterprise procurement pack (Volume XVII) for external audit.

Volume VII

Verification

Chapter 9 — Release Verification

9.1 Cryptographic Manifests

Every release **MUST** include a cryptographic manifest that records the hash of every artifact in the release. The manifest **MUST** use SHA-256 as the primary hash algorithm and SHA-512 as the secondary (for defense against future cryptographic breaks). The manifest format is: one line per artifact, containing the SHA-256 hash, two spaces, and the file path relative to the release root. Paths **MUST** be sorted lexicographically. The manifest **MUST NOT** include itself in its own hash (self-exclusion rule). The manifest **MUST** be generated after all artifacts are finalized and **MUST NOT** be modified after generation. The vres manifest command generates the manifest, and the vres verify command validates it against the actual artifacts.

9.2 Digital Signatures

The cryptographic manifest (checksums.txt) **MUST** be signed by the Release Engineer using the repository signing key. The signature **MUST** be produced using Ed25519 and **MUST** be stored as checksums.sig alongside the manifest. The signature file **MUST** also include the public key or a reference to the key in a trusted key store. Verification of the manifest signature **MUST** be the first step in any release verification workflow — if the signature does not verify, all subsequent verification is unreliable. The vres verify command **MUST** check the manifest signature before checking individual artifact hashes.

9.3 SLSA Provenance

Release artifacts **MUST** include SLSA provenance metadata documenting the build process. The provenance **MUST** record: (a) the builder identity (Sigstore OIDC identity), (b) the source repository and commit, (c) the build configuration, (d) all input artifacts and their hashes, (e) the output artifacts and their hashes, and (f) the build timestamp. Provenance **MUST** be generated by the build platform (not by user-supplied data) and **MUST** be signed by the build platform attestation key. The provenance.json artifact **MUST** conform to the SLSA provenance v0.2 schema. Verification of SLSA provenance **MUST** confirm that the claimed source commit, when built with the claimed configuration, produces the claimed output artifacts (reproducibility verification).

9.4 Machine-Readable Validation

All verification results **MUST** be available in machine-readable JSON format to enable automated downstream verification. The verification.json artifact **MUST** contain: (a) the verification timestamp, (b) the verifier identity, (c) the verification tool and version, (d) each check performed and its result (PASS/FAIL/WARN), (e) the checksum manifest verification result, (f) the signature verification result, (g) the SLSA provenance verification result, and (h) any warnings or informational notes. This artifact enables CI/CD pipelines, procurement systems, and compliance tools to verify releases without human intervention.

9.1.1 Manifest Format Specification

The cryptographic manifest (checksums.txt) **MUST** follow this exact format: one line per artifact, containing the SHA-256 hash in hexadecimal (64 characters), two spaces, and the file path relative to the release root. Lines **MUST** be sorted lexicographically by file path. The manifest **MUST NOT** include itself or its signature file. Blank lines and comments (lines starting with #) are allowed but discouraged. The manifest **MUST** use Unix line endings (LF) and **MUST NOT** have a trailing newline. This format is compatible with the output of sha256sum on Linux systems, enabling verification with standard tools.

```
# checksums.txt — VRES v1.0 Release Manifest
# Generated: 2026-08-06T12:00:00Z
# Product: IVE v1.2.3
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
artifacts/ive-1.2.3-py3-none-any.whl
a7ffc6f8bf1ed76651c14756a061e662f5db255b216e1847e0e8356a7b5e3a4d
artifacts/ive-1.2.3.tar.gz
8928a2c3cb4f929e7e7f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f4f sbom.spdx.json
0a3b1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b sbom.cyclonedx.json
...
```

9.2.1 Signature Verification Procedure

The signature verification procedure is the first step in any release verification workflow. The procedure **MUST** be: (1) Obtain checksums.sig, checksums.crt (certificate), and checksums.txt. (2) Verify the certificate chain: the certificate **MUST** be issued by a trusted CA (Sigstore Fulcio for keyless signing), the certificate identity **MUST** match the expected Release Engineer identity, and the certificate **MUST** not be expired. (3) Verify the signature: the Ed25519 signature in checksums.sig **MUST** verify against checksums.txt using the public key from the certificate. (4) Verify the transparency log entry: the signature **MUST** have a Sigstore Rekor transparency log entry confirming the signature was recorded at the claimed time. If any step fails, the entire verification **MUST** fail and **MUST** report the specific failure. The vres verify command implements this procedure automatically.

9.3.1 SLSA Provenance Schema

The SLSA provenance (provenance.json) **MUST** conform to the SLSA provenance v0.2 schema. The provenance records the complete build process in a machine-readable format that can be verified by any party. The schema requires the following top-level fields: builder (identity of the build platform), buildType (URI identifying the build type), invocation (the build configuration and entry point), materials (input artifacts with hashes), and metadata (build timestamp and other metadata). The provenance **MUST** be signed by the build platform attestation key and **MUST** be verified before the release is promoted.

```
{
"schemaVersion": "0.2",
"builder": {
"id": "https://github.com/vvu/ive/.github/workflows/release.yml"
},
"buildType": "https://slsa-framework.github.io/github-actions-workflow",
"invocation": {
"configSource": {
```

```
"uri": "git+https://github.com/vvu/ive@v1.2.3",
"digest": {"gitCommit": "abc1234..."},
"entryPoint": "release"
},
"parameters": {}
},
"materials": [
{
"uri": "git+https://github.com/vvu/ive@v1.2.3",
"digest": {"gitCommit": "abc1234..."}
}
]
}
```

9.4.1 Verification Command Sequence

The complete verification sequence for a VRES release is as follows. Each step **MUST** pass before the next step is executed. If any step fails, the verification fails and the release **MUST NOT** proceed.

```
# Complete VRES release verification sequence
vres verify --step=manifest-signature # Step 1: Verify manifest signature
vres verify --step=manifest-hashes # Step 2: Verify artifact hashes against manifest
vres verify --step=provenance-signature # Step 3: Verify provenance signature
vres verify --step=provenance-content # Step 4: Verify provenance content
vres verify --step=sbom-completeness # Step 5: Verify SBOM completeness
vres verify --step=license-compatibility # Step 6: Verify license compatibility
vres verify --step=artifact-signatures # Step 7: Verify individual artifact
signatures
vres verify --step=quality-gates # Step 8: Verify quality gate results
vres verify --step=version-consistency # Step 9: Verify version consistency
# Or all at once:
vres verify --all # Execute all verification steps
```

Volume VIII

Documentation

Chapter 10 — Documentation Requirements

10.1 Required Documentation

Every release **MUST** include the following documentation. Documentation **MUST** be versioned alongside the code — it is a release artifact, not an afterthought. Documentation **MUST** be reviewed by the Documentation Freeze gate (Chapter 2, State 6) and **MUST** be complete before a release candidate is built. Documentation for public APIs **MUST** be generated from source code (not hand-written) to ensure accuracy and reduce maintenance burden.

Document	Content	Requirement
README.md	Project overview, quick start, build instructions	MUST
ARCHITECTURE.md	System architecture, component diagram, data flow	MUST

Document	Content	Requirement
SECURITY.md	Security model, reporting process, policy	MUST
API Reference	Public API documentation from source	MUST for public APIs
SDK Documentation	SDK usage, examples, migration guides	MUST for SDK products
DEPLOYMENT.md	Installation, configuration, deployment procedures	MUST
OPERATIONS.md	Monitoring, alerting, runbooks, scaling	MUST
TROUBLESHOOTING.md	Common issues, diagnostics, resolution steps	SHOULD
MIGRATION.md	Version-to-version migration instructions	MUST for breaking changes
DEPRECATION.md	Deprecated features, timeline, alternatives	MUST for deprecations
CHANGELOG.md	Structured changelog (Keep a Changelog format)	MUST
RELEASE_NOTES.md	User-facing changes for this release	MUST

10.2 Documentation Quality

Documentation **MUST** satisfy the following quality requirements: (a) Every public API **MUST** have a description, parameter documentation, return type documentation, and at least one usage example. (b) Architecture documentation **MUST** include a component diagram and **MUST** describe the data flow between components. (c) Deployment documentation **MUST** be tested by executing the documented steps on a clean environment — untested deployment documentation is a defect. (d) All documentation **MUST** be written in the same language as the codebase (English for VVU). (e) Documentation **MUST NOT** contain TODO markers or placeholder content in releases. (f) Broken links in documentation **MUST** be detected and reported by the vres lint command.

10.1.1 README.md Template Content

The README.md **MUST** contain the following sections in order: (1) Product name and one-line description. (2) Badges for build status, coverage, license, and VRES conformance level. (3) Table of contents with links to each section. (4) Installation instructions with prerequisites and platform-specific notes. (5) Quick start guide with a working example. (6) Configuration reference with all configurable parameters. (7) Usage examples for common scenarios. (8) API reference link (generated documentation). (9) Contributing guide link. (10) License and copyright notice. The README **MUST** be the entry point for a new user and **MUST** provide sufficient information to install and use the product without consulting additional documentation. The vres lint --readme command **MUST** verify that all required sections are present and that all links are valid.

10.1.2 ARCHITECTURE.md Template Content

The ARCHITECTURE.md **MUST** contain: (1) System overview with a component diagram (embedded as Mermaid or PlantUML). (2) Component descriptions with responsibilities and interfaces. (3) Data flow diagram showing how data moves through the system. (4) Dependency map showing internal and external dependencies. (5) Deployment architecture for each supported topology (Volume XI). (6) Security architecture with trust boundaries and data classification. (7) Scalability design with scaling factors and bottlenecks. (8) Extension points for product overlays (Volume XIX). The architecture document **MUST** be updated when the system architecture changes and **MUST** be reviewed by the Release Engineer before any major release.

10.2.1 Documentation Review Checklist

Every document in the release **MUST** pass the following review checklist before the Documentation Freeze gate (Chapter 2, State 6) passes. The checklist is evaluated by the vres lint --docs command and by human review.

Check	Description	Req.	Method
D01	All required documents present (Section 10.1)	MUST	Automated
D02	No TODO/FIXME/placeholder markers	MUST	Automated
D03	All links valid (no 404s)	MUST	Automated
D04	Code examples compile/run	MUST	Automated
D05	Version numbers consistent with release	MUST	Automated
D06	API documentation matches source code	MUST	Automated
D07	No forbidden terms (Section 1.2.5.2)	MUST	Automated
D08	Spelling and grammar check passed	SHOULD	Automated
D09	Technical accuracy reviewed by SME	MUST	Human
D10	Accessibility requirements met	SHOULD	Manual

10.2.2 API Documentation Generation

API documentation **MUST** be generated from source code to ensure accuracy. Python products **MUST** use Sphinx with autodoc to generate API documentation from docstrings. TypeScript products **MUST** use TypeDoc to generate API documentation from TSDoc comments. Rust products **MUST** use rustdoc to generate API documentation from doc comments. The generated documentation **MUST** include: function signatures with type annotations, parameter descriptions, return type descriptions, exception/error descriptions, usage examples, and cross-references to related functions. The API documentation **MUST** be generated as part of the CI/CD pipeline and **MUST** be published to the product documentation site. The generated output **MUST** be validated for completeness — every public API **MUST** have documentation, and the vres lint --api-docs command **MUST** report any undocumented public APIs.

Volume IX

Release Artifacts

Chapter 11 — Release Artifact Manifest

11.1 Required Artifacts

Every release **MUST** contain the following artifacts. No artifact **MAY** be omitted without a documented exception approved by the Release Engineer. The artifact set is designed to provide complete auditability (Volume XV), verification (Volume VII), and procurement readiness (Volume XVII) from a single release bundle.

Artifact	Description	Req.	Format
LICENSE	License text for the product	MUST	Plain text
NOTICE	Third-party notices and attributions	MUST	Plain text
CHANGELOG.md	Structured changelog	MUST	Markdown
RELEASE_NOTES.md	User-facing release notes	MUST	Markdown
SBOM (SPDX)	Software bill of materials (SPDX format)	MUST	JSON
SBOM (CycloneDX)	Software bill of materials (CycloneDX format)	MUST	JSON
checksums.txt	SHA-256 manifest of all artifacts	MUST	Text
checksums.sig	Ed25519 signature of checksums.txt	MUST	Binary
release.json	Release metadata (version, date, commmit)	MUST	JSON
provenance.json	SLSA provenance for the build	MUST	JSON
manifest.json	Artifact manifest with hashes	MUST	JSON
verification.json	Verification results	MUST	JSON
test-report.json	Test execution results and coverage	MUST	JSON
security-report.json	Security scan results	MUST	JSON
compliance-report.json	License and compliance audit	MUST	JSON
signatures/	Directory of individual artifact signatures	MUST	Directory
artifacts/	Directory of distributable artifacts	MUST	Directory

11.2 release.json Schema

The release.json artifact **MUST** conform to the following schema. It provides machine-readable release metadata for automated systems and must be generated by the vres release command.

```
{
"schema_version": "1.0.0",
"release_id": "v1.2.3",
"release_type": "major|minor|patch|hotfix|pre-release",
"source_commit": "abc1234def567890...",
"branch": "main",
"timestamp": "2026-08-06T12:00:00Z",
"builder_identity": "sigstore:oidc:...",
"products": ["ive", "hbk-mkii"],
"quality_gates": { "all_passed": true, "details": [...] },
"license_status": "VALIDATED",
"sbom_hash_sha256": "...",
"provenance_hash_sha256": "...",
"signatures_verified": true
}
```

11.2.1 provenance.json Schema

The provenance.json artifact **MUST** conform to the SLSA provenance v0.2 schema as detailed in Section 9.3.1. This artifact provides machine-readable proof of how the release was built, enabling downstream consumers to verify the build process without trusting the publisher alone.

11.2.2 verification.json Schema

The verification.json artifact records the complete verification results for the release. It **MUST** be generated by the vres verify command and **MUST** be included in the release artifacts.

```
{
"schema_version": "1.0.0",
"release_id": "v1.2.3",
"verification_timestamp": "2026-08-06T12:30:00Z",
"verifier_identity": "vres-verify/1.0.0",
"checks": {
"manifest_signature": {"result": "PASS", "detail": "Ed25519 signature verified"},
"manifest_hashes": {"result": "PASS", "detail": "All 17 artifacts verified"},
"provenance_signature": {"result": "PASS", "detail": "SLSA provenance verified"},
"provenance_content": {"result": "PASS", "detail": "Source commit matches"},
"sbom_completeness": {"result": "PASS", "detail": "142 dependencies enumerated"},
"license_compatibility": {"result": "PASS", "detail": "All licenses compatible"},
"artifact_signatures": {"result": "PASS", "detail": "3 signed artifacts verified"},
"quality_gates": {"result": "PASS", "detail": "19/19 gates passed"},
"version_consistency": {"result": "PASS", "detail": "v1.2.3 in all locations"}
},
"overall_result": "PASS"
}
```

11.2.3 security-report.json Schema

The security-report.json artifact records the results of all security scans performed during the release process. It **MUST** include: SAST findings with severity and location, DAST findings with severity and request/response details, dependency vulnerability scan results with CVE identifiers and CVSS scores, container scan results for each scanned image, and secret scan results. The security report **MUST** be reviewed by the Security Officer before release and **MUST** be included in the enterprise procurement pack (Volume XVII) for external audit.

11.3 Artifact Generation Procedure

The artifact generation procedure **MUST** be automated and deterministic. The vres release command orchestrates the following sequence: (1) Build all product artifacts (binaries, containers, documentation). (2) Generate SBOM in SPDX and CycloneDX formats from lockfiles. (3) Generate test-report.json from test execution results. (4) Generate security-report.json from scan results. (5) Generate compliance-report.json from license audit. (6) Generate release.json with release metadata. (7) Generate provenance.json with build provenance. (8) Generate checksums.txt manifest of all artifacts. (9) Sign checksums.txt and critical artifacts. (10) Generate verification.json from verification results. (11) Package all artifacts into the release bundle. Each step **MUST** be idempotent — running the command multiple times with the same inputs **MUST** produce identical outputs.

11.4 Artifact Verification Procedure

The artifact verification procedure is executed by downstream consumers to verify the integrity and authenticity of the release. The procedure is: (1) Download the release bundle. (2) Verify the manifest signature (checksums.sig against checksums.txt). (3) Verify each artifact hash against the manifest. (4) Verify SLSA provenance signature and content. (5) Verify SBOM completeness. (6) Verify license compatibility. (7) Verify individual artifact signatures for critical artifacts. (8) Verify quality gate results. (9) Verify version consistency. If all steps pass, the release is verified. The vres verify --all command automates this procedure. For air-gapped environments, the procedure **MUST** be executable offline using vendored verification data.

Volume X

Compliance

Chapter 12 — Compliance Requirements

12.1 License Compliance

All dependencies **MUST** be classified according to the license compatibility matrix (Section 6.3). The compliance-report.json artifact **MUST** include: (a) the total number of dependencies, (b) each dependency with its license and compatibility status, (c) any exceptions with justification and approver, and (d) a summary of compatible, conditional, and incompatible licenses. The Compliance Officer **MUST** review and approve the compliance report before release. No dependency with an incompatible license **MAY** be included in a VVU product without a formal exception documenting the business justification, risk assessment, and mitigation plan.

12.2 Open Source Compliance

VVU products that include or distribute open-source components **MUST** comply with the license terms of those components. This includes: (a) retaining all license and notice files, (b) providing attribution as required by the license, (c) making source code available where required by copyleft licenses, and (d) documenting modifications to open-source components. The NOTICE file **MUST** list all third-party components with their licenses and copyright holders. For the AGPL-3.0-or-later track, VVU **MUST** provide the complete corresponding source code for any network-accessible instance of the product.

12.3 Export Control

VVU products that include cryptographic functionality **MUST** be reviewed for export control compliance. Cryptographic modules (Ed25519 signing, ZK-proof generation, MMR construction) **MUST** be documented with their algorithm identifiers, key sizes, and implementation sources. For products distributed internationally, the export classification (ECCN) and any applicable license exceptions **MUST** be documented in the compliance report. HBK Mk-II firmware that includes cryptographic signing **MUST** be classified under the applicable Wassenaar Arrangement provisions for South African export.

12.4 Academic Publication Policy

VVU research artifacts (HBK Mk-II hydraulic simulations, experimental datasets) intended for academic publication **MUST** include: (a) a reproducibility package with all data, code, and configuration necessary to reproduce the published results, (b) a DORA (Declaration on Research Assessment) compliant data availability statement, (c) explicit licensing of research data (recommended: CC-BY-4.0 for data, CC0 for metadata), and (d) a citation file (CITATION.cff) following the Citation File Format specification. The reproducibility package **MUST** be validated by an independent researcher before publication, and the validation result **MUST** be included in the evidence package.

12.1.1 License Scanning Procedure

License scanning **MUST** be performed on every pull request that modifies lockfiles and on every release candidate. The scanning procedure is: (1) The SBOM is generated from the lockfile (Section 6.1). (2) Each dependency in the SBOM is looked up in the license database (SPDX License List + VVU custom entries). (3) Each license is classified according to the compatibility matrix (Section 6.3). (4) Dependencies with incompatible or conditional licenses are flagged for review. (5) The compliance report is generated with the full classification. (6) If any incompatible licenses are found, the Compliance Officer **MUST** approve an exception. The scanning tool **MUST** be updated with new license identifiers as they are added to the SPDX License List. The vres lint --licenses command **MUST** report the license classification for every dependency in the product.

12.3.1 Export Control Classification

VVU products include cryptographic functionality that may be subject to export control. The following table classifies the cryptographic components and their export status.

Component	Product	Algorithm	ECCN	Status
Ed25519 Signing	Sigstore/Cosign	ECC, 256-bit	EAR99	Publicly available
ZK-Proof Generation	ProofBridge	Groth16, BN254	5D002	Review required
MMR Construction	ProofBridge	Hash (SHA-256)	EAR99	Not encryption
AES-GCM Encryption	Trust Sphere	Symmetric, 256-bit	5D002	Review required
RSA Key Operations	SafeKrypte	RSA, 4096-bit	5D002	Review required
Sensor Calibration	HBK Mk-II	No cryptography	EAR99	N/A

12.1.2 Third-Party Notice Format

The NOTICE file **MUST** list all third-party components with their licenses and copyright holders. The format is: for each component, the component name and version, the copyright holder(s), the license identifier (SPDX short form), and the full license text (if required by the license terms). Components under permissive licenses (MIT, BSD, Apache-2.0) **MUST** be listed with attribution. Components under copyleft licenses (GPL, AGPL) **MUST** include the complete license text and a statement of where the corresponding source code is available. The NOTICE file **MUST** be generated automatically from the SBOM and **MUST** be reviewed by the Compliance Officer before each release.

12.4.1 Academic Reproducibility Package

The academic reproducibility package **MUST** contain: (a) The complete dataset used in the study, in an open format (CSV, Parquet, or HDF5 with documented schema). (b) The analysis code with exact version pinning (via lockfile) and a README describing how to execute the analysis. (c) The environment specification: operating system, runtime versions, hardware specification (CPU, GPU, memory), and any required specialized hardware. (d) The validation results with statistical analysis, confidence intervals, and comparison against the Brier Score threshold. (e) A CITATION.cff file following the Citation File Format specification. (f) A DORA-compliant data availability statement. (g) A reproducibility certificate signed by the independent researcher who verified the results. The package **MUST** be cryptographically sealed to prevent post-publication tampering.

Volume XI

Enterprise Deployment

Chapter 13 — Deployment Models

13.1 Supported Deployment Topologies

VVU products **MUST** support the following deployment topologies. Each topology has specific requirements for installation, configuration, monitoring, and disaster recovery. Deployment documentation (Volume VIII) **MUST** cover each supported topology with tested procedures.

Topology	Use Case	Architecture	Criticality
Single Node	Development, small team	SQLite, single process	Low
High Availability	Production, team	PostgreSQL, replicated, load-balanced	Medium
Air-Gapped	Government, defense	Offline install, vendored deps, local registry	High
Government	Municipal, national	Air-gapped + compliance + audit export	Critical
Research Cluster	University, HPC	SLURM integration, ROCm/CUDA, batch processing	High
Cloud	SaaS, PaaS	Kubernetes, managed services, auto-scaling	Medium
Hybrid	Enterprise	Cloud + on-premises, data synchronization	High

Topology	Use Case	Architecture	Criticality
Edge	IoT, field	ARM64, minimal footprint, OTA updates	High
HBK Appliance	Hydraulic research	Pre-configured, HIL-validated, calibration-locked	Critical
IVE Appliance	Verification platform	Pre-configured, evidence-locked, audit-ready	Critical

13.2 Air-Gapped Deployment

Air-gapped deployment is a critical requirement for VVU products targeting South African government and municipal infrastructure. The air-gapped deployment package **MUST** include: (a) all application artifacts with signatures, (b) all dependency artifacts (vendored), (c) all container base images, (d) installation media with SHA-256 verification, (e) offline documentation bundle, (f) a vres doctor --offline verification report, and (g) a deployment checklist specific to the air-gapped topology. The installation process **MUST** verify all checksums and signatures before any artifact is deployed, and **MUST** refuse to proceed if any verification fails.

13.1.1 High Availability Installation Procedure

The High Availability (HA) topology installation procedure **MUST** produce a deployment with no single point of failure. The procedure is: (1) Provision three or more nodes in different availability zones. (2) Install PostgreSQL with streaming replication (primary + two standbys). (3) Install the application on each node with identical configuration. (4) Configure a load balancer (HAProxy or cloud LB) with health check endpoints. (5) Configure session affinity if required by the application. (6) Verify that failover works by terminating the primary node and confirming service continues on the standby. (7) Verify that data replication is working by writing a record on the primary and reading it on the standby. (8) Document the deployment in the operations runbook. The entire procedure **MUST** be automated using infrastructure-as-code (Terraform/Pulumi) and **MUST** be idempotent.

13.1.2 Air-Gapped Installation Procedure

The air-gapped installation procedure is critical for South African government and municipal deployments. The procedure is: (1) On the online preparation machine, run vres release --airgap-pack to generate the air-gapped deployment bundle containing all artifacts, vendored dependencies, container images (as tar archives), and installation scripts. (2) Transfer the bundle to the air-gapped environment via approved physical media (encrypted USB, DVD) or approved network transfer protocol. (3) On the air-gapped machine, verify the bundle integrity: check SHA-256 hashes of all files against the manifest, verify digital signatures, and verify the SBOM. (4) Execute the installation script, which loads container images from tar, installs vendored dependencies, and configures the application for offline operation. (5) Run vres doctor --offline to verify the installation. (6) Document the installation with the deployment checklist specific to the air-gapped topology.

13.2.1 Government Topology Requirements

The government topology adds compliance and audit requirements on top of the air-gapped topology. In addition to the air-gapped requirements, the government topology MUST: (a) Support audit log export in the format required by the South African Public Finance Management Act (PFMA) and the Municipal Finance Management Act (MFMA). (b) Implement role-based access control aligned with the government security classification framework. (c) Provide a compliance dashboard showing current status against applicable regulatory requirements (POPIA, PFMA, MFMA, B-BBEE). (d) Support integration with government identity providers (e.g., SAFRA, departmental LDAP). (e) Provide data sovereignty guarantees: all data **MUST** remain within South African borders, with no cross-border data transfer without explicit authorization. (f) Include a security assessment report from an approved South African security assessment provider.

13.3 Configuration Management

Configuration for all deployment topologies **MUST** be managed through version-controlled configuration files, not through manual configuration or environment variables. The configuration hierarchy is: (1) Default configuration (in the product codebase). (2) Environment configuration (per topology: dev, staging, production). (3) Instance configuration (per deployment instance, for overrides). Later layers override earlier layers, and the effective configuration is the merge of all layers. Configuration files **MUST** be validated against a schema before the application starts. Invalid configuration **MUST** cause the application to fail to start with a clear error message identifying the invalid field. Secrets **MUST NOT** be stored in configuration files — they **MUST** be provided through a secrets manager (HashiCorp Vault, AWS Secrets Manager, or Kubernetes Secrets with encryption at rest).

Volume XII

Operational Excellence

Chapter 14 — Operational Requirements

14.1 Monitoring and Observability

Deployed VVU products **MUST** expose health checks, metrics, and structured logs. Health checks **MUST** include: (a) a liveness check (is the process running?), (b) a readiness check (can the process serve requests?), and (c) a startup check (has initialization completed?). Metrics **MUST** be exposed in OpenMetrics format and **MUST** include: request latency, error rate, throughput, resource utilization, and product-specific metrics (e.g., for HBK Mk-II: sensor reading rate, calibration status, Brier Score; for ProofBridge: receipt issuance rate, MMR depth). Structured logs **MUST** use JSON format with correlation IDs for request tracing. All observability data **MUST** be retained for a minimum of 90 days for operational analysis and incident investigation.

14.2 Incident Response

The incident response process **MUST** define: (a) severity levels (SEV1: data loss or safety risk, SEV2: service degradation, SEV3: minor impact, SEV4: informational), (b) response time targets (SEV1: 15 minutes, SEV2: 1 hour, SEV3: 4 hours, SEV4: next business day), (c) escalation paths, (d) communication templates, and (e) post-incident review procedures. Every SEV1 and SEV2 incident **MUST** have a post-incident review documenting root cause, timeline, and preventive measures. Incident reports **MUST** be retained for the lifetime of the product.

14.3 Release Rollback

Every release **MUST** have a documented and tested rollback procedure. The rollback procedure **MUST** restore the previous known-good version and **MUST** be completable within the product RTO (Volume XVIII). Database migrations that are not reversible **MUST** be documented with manual rollback steps. The vres rollback command **MUST** automate the rollback procedure where possible and **MUST** produce a rollback report documenting the previous version, target version, rollback timestamp, and any manual steps required. Rollback testing **MUST** be part of the release verification process for every major and minor release.

14.4 Hotfix Strategy

Hotfixes follow an abbreviated release lifecycle: Development → Candidate Build → Verification (critical gates only) → Signing → Release. The abbreviated lifecycle **MUST** still include: security scanning, test execution for affected areas, artifact signing, and manifest generation. Hotfixes **MUST NOT** include new features — they are limited to defect fixes and security patches. A hotfix release **MUST** be followed by a full release within 30 days that brings the hotfix through the complete lifecycle. Hotfix commits **MUST** be cherry-picked to the main branch and **MUST** go through the standard review process after initial emergency release.

14.1.1 Monitoring Stack Details

The recommended monitoring stack for VVU deployments consists of: Prometheus for metrics collection and alerting, Grafana for dashboards and visualization, Loki for log aggregation, Tempo for distributed tracing, and AlertManager for alert routing and notification. This stack is selected for its open-source licensing, active community, and compatibility with the OpenMetrics format used by VVU products. For government deployments where open-source monitoring may not be approved, the product **MUST** also support emitting metrics in formats compatible with commercial monitoring tools (Datadog, New Relic, Splunk) through sidecar exporters. The monitoring stack **MUST** be deployed as part of the infrastructure-as-code provisioning and **MUST** be configured with product-specific dashboards and alert rules.

14.2.1 Incident Response Playbook

The incident response playbook defines the step-by-step procedure for handling each severity level. For SEV1 (Critical): (1) Acknowledge the incident within 15 minutes. (2) Assign an incident commander. (3) Open a war room (Slack channel or physical room). (4) Assess the scope: which products, deployments, and users are affected. (5) Implement containment: if a released artifact is compromised, mark it as vulnerable in the artifact repository. (6) Develop a fix following the hotfix process (Section 14.4). (7) Release the fix. (8) Verify the fix resolves the incident. (9) Conduct a post-incident review within 5 business days. (10) Publish the post-incident report (after redacting sensitive information). For SEV2-SEV4, the playbook follows the same structure with relaxed time requirements.

14.3.1 Rollback Procedure Details

The rollback procedure **MUST** be tested for every major and minor release. The procedure is: (1) The Release Engineer invokes vres rollback --target PREVIOUS_VERSION. (2) The system verifies that the target version artifacts exist and are signed. (3) The system stops the current version (graceful shutdown with drain period). (4) The system deploys the target version artifacts. (5) The system verifies the target version health check passes. (6) The system restores the database to the state compatible with the target version (if migration rollback is required). (7) The system restores traffic to the target version. (8) The system monitors for 30 minutes to confirm stability. (9) The rollback report is generated and recorded in the audit trail. Database migrations that are not reversible **MUST** be documented with manual rollback steps and **MUST** be tested during the release verification process.

14.4.1 Hotfix Procedure Details

The hotfix procedure follows an abbreviated lifecycle: (1) Create a hotfix branch from the release tag (hotfix/vX.Y.Z+1). (2) Implement the fix with minimal scope — no refactoring, no new features, no documentation updates beyond the essential fix description. (3) Execute the abbreviated test suite: unit tests for affected modules, integration tests for affected paths, security scan, and HIL tests (for HBK Mk-II). (4) Build the hotfix release candidate. (5) Sign and verify the hotfix artifacts. (6) Release the hotfix with a clear description of the fix and the CVE or defect identifier. (7) Cherry-pick the fix to the main branch and go through the standard review process. (8) Schedule a full release within 30 days that includes the hotfix and any deferred items. The hotfix procedure **MUST** be documented in the operations runbook and **MUST** be rehearsed during the release verification process.

Volume XIII

Quality Gates

Chapter 15 — Quality Gate Specification

15.1 Mandatory Quality Gates

A release **MUST NOT** proceed unless ALL of the following quality gates pass. Each gate is evaluated by an automated process and produces a PASS or FAIL result. WARN results do not block the release but **MUST** be documented. The quality gate results are recorded in verification.json and in the audit trail (Volume XV). The vres verify command evaluates all gates and produces a summary report.

Gate	Name	Criteria	Req.	Eval
G01	Formatting	Code style matches configured formatter	MUST	Automated
G02	Lint	No lint errors at configured severity	MUST	Automated
G03	Type Safety	Type checker passes (TypeScript strict, mypy)	MUST	Automated
G04	Static Analysis	SAST tool passes with no high/critical findings	MUST	Automated
G05	Unit Tests	All unit tests pass, coverage meets threshold	MUST	Automated
G06	Integration Tests	All integration tests pass	MUST	Automated
G07	End-to-End Tests	All E2E tests pass against release candidate	MUST	Automated
G08	Security Scan	No critical/high CVEs in dependencies	MUST	Automated
G09	Secret Scan	No secrets detected in code or artifacts	MUST	Automated
G10	SBOM	SBOM generated and validated against dependency tree	MUST	Automated
G11	License Audit	All dependencies compatible per Section 6.3	MUST	Automated
G12	Manifest Verification	Cryptographic manifest matches artifacts	MUST	Automated
G13	Digital Signature	All required artifacts signed and verified	MUST	Automated
G14	Documentation	All required documents present and reviewed	MUST	Review

Gate	Name	Criteria	Req.	Eval
G15	Performance Budget	No regression > 5% vs baseline	MUST	Automated
G16	Packaging	All required artifacts present per Section 11.1	MUST	Automated
G17	Release Notes	Release notes present and accurate	MUST	Review
G18	Compliance	Compliance report reviewed and approved	MUST	Review
G19	Approval Workflow	All required approvals obtained	MUST	Process

15.2 Gate Evaluation Order

Quality gates **MUST** be evaluated in the order listed above. Early gates (formatting, lint, type safety) are fast and inexpensive — they **SHOULD** be evaluated on every pull request. Later gates (E2E tests, performance budget, packaging) are slower and more expensive — they **SHOULD** be evaluated on release candidates only. The evaluation order enables fast feedback: a developer receives formatting errors within seconds, not after a 30-minute end-to-end test suite. Gates that fail **MUST** produce actionable error messages that identify the specific failure and suggest remediation.

15.1.1 Gate G01: Formatting — Detailed Specification

Gate G01 verifies that all source code matches the configured formatter output. The formatter is applied to all source files and the result is compared against the committed code. Any difference is a failure. Python products **MUST** use Ruff (format mode) with line-length 100. TypeScript products **MUST** use Prettier with semi: true, singleQuote: true, printWidth: 100. Rust products **MUST** use rustfmt with max_width = 100. Go products **MUST** use gofmt. C++ products **MUST** use clang-format with the Google style as base. The formatter configuration **MUST** be committed to the repository and **MUST NOT** be changed without Release Engineer approval. Pre-commit hooks **MUST** be configured to run the formatter on every commit. If G01 fails, the remediation is to run the formatter and commit the result — no manual formatting review is required.

15.1.2 Gate G08: Security Scan — Detailed Specification

Gate G08 verifies that no critical or high severity vulnerabilities exist in the product dependencies. The scan consults the National Vulnerability Database (NVD), GitHub Advisory Database, and OSV database. The scan **MUST** be performed using: pip-audit for Python, cargo-audit for Rust, npm audit for TypeScript, and Trivy for containers. For each vulnerability found, the gate records the CVE identifier, the CVSS score, the affected package and version, and the available remediation (upgrade version or patch). Critical (CVSS >= 9.0) and High (CVSS >= 7.0) vulnerabilities **MUST** block the release. Medium vulnerabilities **SHOULD** be remediated but **MAY** be accepted with documented justification and Security Officer approval. If G08 fails, the remediation is to upgrade the affected dependency to a version without the vulnerability, or to obtain a Security Officer exception with documented justification.

15.2.1 Gate Failure Handling

When a quality gate fails, the following handling procedure **MUST** be followed: (1) The gate failure is recorded in the audit trail with the gate identifier, failure details, and timestamp. (2) The CI/CD pipeline stops — subsequent gates do not execute. (3) The responsible role is notified based on the gate category: code quality gates notify the developer, security gates notify the Security Officer, compliance gates notify the Compliance Officer. (4) The failure details include specific, actionable information: the file and line for lint errors, the CVE identifier and CVSS score for security findings, the missing artifact for packaging failures. (5) The developer remediates the failure and re-submits the change. (6) The pipeline re-executes from the failed gate (not from the beginning, unless earlier gates are affected). Exceptions to gate failures **MUST** be documented with the exception authority, justification, and expiry (72 hours maximum). Expired exceptions **MUST** be re-evaluated.

15.2.2 Gate Remediation Guidance

Each quality gate **MUST** provide specific remediation guidance when it fails. The guidance **MUST** include: the specific command to fix the issue (e.g., ruff format . for G01), the expected outcome after remediation, and a link to the relevant VRES section for detailed requirements. The remediation guidance **MUST** be included in the CI/CD pipeline output as both human-readable text and machine-readable JSON. For automated gates, the remediation command **SHOULD** be executable by the developer without Release Engineer involvement. For review gates (G14, G17, G18), the remediation is to complete the required review and obtain the required approval. For process gates (G19), the remediation is to obtain the missing approval from the designated authority. The vres verify --remediate command **MUST** output the remediation guidance for any failed gates.

Volume XIV

Continuous Delivery

Chapter 16 — CI/CD Pipeline Requirements

16.1 Supported CI Platforms

VRES defines CI/CD pipeline templates for the following platforms. Products **MAY** use any supported platform but **MUST** implement the same quality gates regardless of platform. Pipeline templates **MUST** be included in the reference repository (Volume II, Section 4.6) and **MUST** be documented with platform-specific configuration instructions.

Platform	Status	Key Features	Config Path
GitHub Actions	Primary	SLSA 3 native via OIDC, Sigstore integration	.github/workflows/
GitLab CI	Supported	Pipeline-as-code, container registry	.gitlab-ci.yml
Azure DevOps	Supported	Enterprise, Azure integration	azure-pipelines.yml
Jenkins	Supported	Legacy, self-hosted	Jenkinsfile
Buildkite	Supported	Elastic, fast	.buildkite/pipeline.yml

16.2 Pipeline Structure

CI/CD pipelines **MUST** implement the following stages, corresponding to the quality gates (Volume XIII): (1) Validate: formatting, lint, type safety, static analysis (G01-G04). (2) Test: unit, integration, regression tests (G05-G07). (3) Security: dependency scanning, secret scanning, container scanning (G08-G09). (4) Package: SBOM generation, license audit, artifact packaging (G10-G11, G16). (5) Verify: manifest verification, signature verification (G12-G13). (6) Release: documentation check, performance check, compliance check, approval workflow (G14-G15, G17-G19). Each stage **MUST** gate on the previous — if stage N fails, stage N+1 **MUST NOT** execute. The pipeline **MUST** produce a test-report.json and verification.json as pipeline artifacts regardless of pass/fail status.

16.3 Self-Hosted Runners

For air-gapped and high-security environments, self-hosted runners **MUST** be supported. Self-hosted runners MUST: (a) run on hardened operating systems with minimal attack surface, (b) be ephemeral (created fresh for each job or recycled on a schedule), (c) have no persistent access to secrets or credentials between jobs, (d) log all job execution details for audit, and (e) be registered with the CI platform using short-lived credentials that are rotated on schedule. For HBK Mk-II HIL testing, self-hosted runners with hardware access **MUST** be isolated in a separate network segment with controlled access to test equipment.

16.1.1 GitHub Actions Pipeline Template

The following GitHub Actions workflow template implements the VRES CI/CD pipeline for a Python product. The template **MUST** be customized for each product but **MUST** implement all required quality gates (Volume XIII).

```
name: VRES Release Pipeline
on:
push:
branches: [main]
tags: ['v*']
pull_request:
branches: [main]

permissions:
id-token: write # SLSA/OIDC
contents: read

jobs:
validate: # G01-G04: Formatting, Lint, Type, SAST
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: astral-sh/setup-uv@v4
- run: uv run ruff format --check .
- run: uv run ruff check .
- run: uv run mypy src/
- run: uv run bandit -r src/

test: # G05-G07: Unit, Integration, E2E
runs-on: ubuntu-latest
needs: validate
steps:
- uses: actions/checkout@v4
- run: uv run pytest --cov -m unit
- run: uv run pytest -m integration
- run: uv run pytest -m e2e
```

```
security: # G08-G09: Dependency scan, Secret scan
runs-on: ubuntu-latest
needs: validate
steps:
- uses: actions/checkout@v4
- run: uv run pip-audit
- uses: TruffleSecurity/trufflehog@main

package: # G10-G11, G16: SBOM, License, Artifacts
runs-on: ubuntu-latest
needs: [test, security]
steps:
- uses: actions/checkout@v4
- run: vres sbom
- run: vres lint --licenses

verify: # G12-G13: Manifest, Signature
runs-on: ubuntu-latest
needs: package
steps:
- uses: actions/checkout@v4
- run: vres manifest
- run: vres sign
- run: vres verify --all
```

16.1.2 GitLab CI Template

The GitLab CI template implements the same pipeline structure as the GitHub Actions template but uses GitLab CI syntax. The template **MUST** be located at .gitlab-ci.yml in the repository root. GitLab CI provides native support for container registry, environment management, and review apps, which **MAY** be used for deployment staging. The template **MUST** use GitLab CI environments with protection rules to ensure that production deployments require manual approval. The template **MUST** use GitLab CI variables (masked and protected) for secrets, and **MUST NOT** include secrets in the pipeline configuration. The template **MUST** generate the same artifacts as the GitHub Actions pipeline: test-report.json, security-report.json, compliance-report.json, and verification.json.

16.3.1 Self-Hosted Runner Configuration

Self-hosted runners **MUST** be configured with the following security requirements: (a) The runner host **MUST** run a supported LTS operating system (Ubuntu 22.04 LTS or 24.04 LTS). (b) The runner **MUST** be ephemeral — use GitHub Actions runner scaling (Actions Runner Controller for Kubernetes) to create fresh runners for each job. (c) The runner **MUST NOT** have persistent access to secrets between jobs — secrets are injected per-job and are not retained. (d) The runner **MUST** log all job execution details (start time, end time, exit code, artifacts produced) to the audit trail. (e) The runner registration token **MUST** be rotated on schedule (quarterly) and on event (runner compromise, token leak). (f) For HIL testing (HBK Mk-II), the runner **MUST** be in a separate network segment with controlled access to test hardware. The runner configuration **MUST** be managed through infrastructure-as-code and **MUST** be documented in the operations runbook.

16.4 Promotion Pipeline

The promotion pipeline manages the progression of a release candidate through the lifecycle states (Chapter 2). The pipeline is: (1) A release tag triggers the promotion pipeline. (2) The pipeline builds the release candidate in a hermetic environment. (3) The pipeline evaluates all quality gates (G01-G19). (4) If all gates pass, the pipeline signs the artifacts and publishes them to the staging repository. (5) The pipeline notifies the Release Engineer that the release is ready for promotion. (6) The Release Engineer reviews the verification results and approves the promotion. (7) The pipeline publishes the release to the production repository. (8) The pipeline generates the enterprise procurement pack (Volume XVII). (9) The pipeline records the release in the audit trail. The promotion pipeline **MUST** be manual-approval-gated for production publication — automatic publication to production is prohibited.

Volume XV

Audit Framework

Chapter 17 — Audit Records

17.1 Audit Record Structure

Every release **MUST** produce an immutable audit record. The audit record is the single source of truth for what was built, how it was built, who built it, and whether it passed all required gates. Audit records **MUST** be append-only — they **MUST NOT** be modified after creation. Audit records **MUST** be retained for the lifetime of the product plus a minimum of seven years. The audit record **MUST** contain the following fields:

Field	Type	Description	Example
release_id	string	Semantic version identifier	v1.2.3
git_sha	string (40 hex)	Source commit hash	abc1234...
builder_identity	string	OIDC identity of builder	sigstore:oidc:...
timestamp	ISO 8601	Build completion time	2026-08-06T12:00:00Z
compiler_versions	object	Language compiler versions used	{"python": "3.12", ...}
os	string	Build operating system	ubuntu-22.04
architecture	string	Target architecture	linux/amd64
dependencies_hash	string	SHA-256 of lockfile	def5678...
sbom_hash	string	SHA-256 of SBOM	ghi9012...
manifest_hash	string	SHA-256 of checksums.txt	jkl3456...
signature_hash	string	SHA-256 of checksums.sig	mno7890...
verification_result	object	Quality gate results	{"all_passed": true, ...}
approvals	array	Approval records with identity and timestamp	[...]

17.2 Audit Immutability

Audit records **MUST** be stored in an append-only store. Modifications to existing audit records **MUST** be technically prevented, not merely policy-prohibited. The recommended implementation is a ledger with cryptographic chain integrity (hash-chained entries), as implemented in the VVU Trust Sphere. For products not yet integrated with Trust Sphere, audit records **MUST** be stored in a write-once storage medium (e.g., S3 with Object Lock, WORM tape) and **MUST** be periodically verified against the known-good state. The vres audit command **MUST** verify the integrity of the audit store and **MUST** report any modifications or gaps in the audit chain.

17.1.1 Audit Record JSON Schema

The audit record **MUST** be stored as a JSON document conforming to the following schema. The schema ensures that all required fields are present and correctly typed, enabling automated audit verification and export.

```
{
"schema_version": "1.0.0",
"audit_id": "uuid-v4",
"release_id": "v1.2.3",
"git_sha": "40-hex-chars",
"builder_identity": "sigstore:oidc:email@vvu.org",
"timestamp": "2026-08-06T12:00:00Z",
"compiler_versions": {
"python": "3.12.4",
"rust": "1.80.0",
"typescript": "5.5.0"
},
"build_environment": {
"os": "ubuntu-22.04",
"architecture": "linux/amd64",
"container_image": "python:3.12-slim@sha256:..."
},
"dependencies_hash": "sha256-of-lockfile",
"artifacts": {
"checksums_hash": "sha256-of-checksums.txt",
"signature_hash": "sha256-of-checksums.sig",
"sbom_hash": "sha256-of-sbom.spdx.json",
"provenance_hash": "sha256-of-provenance.json"
},
"quality_gates": {
"all_passed": true,
"details": [
{"gate": "G01", "result": "PASS"},
{"gate": "G02", "result": "PASS"}
]
},
"approvals": [
{
"approver": "release-engineer@vvu.org",
"timestamp": "2026-08-06T12:30:00Z",
"type": "release_approval"
}
]
}
```

17.2.1 Retention Policy

Audit records **MUST** be retained according to the following schedule. The retention period starts from the release date and extends for the specified duration. Records **MUST NOT** be deleted before the retention period expires, even if the product reaches end of life. Retention periods are based on regulatory requirements (PFMA requires 5 years, MFMA requires 5 years, POPIA requires records of processing activities), industry best practices (7 years for financial records), and VVU policy (lifetime of product + 7 years for safety-critical).

Record Type	Retention Period	Rationale
Release audit records	Product lifetime + 7 years	Safety-critical requirement
Security scan results	Product lifetime + 7 years	Incident investigation
Test execution results	5 years	Regression analysis
CI/CD pipeline logs	3 years	Build reproducibility
Approval records	Product lifetime + 7 years	Compliance audit
Incident reports	Product lifetime + 7 years	Legal and safety
Compliance reports	Product lifetime + 5 years	Regulatory audit
SBOM documents	Product lifetime + 7 years	Vulnerability tracking

17.3 Audit Verification

The vres audit --verify command **MUST** verify the integrity of the audit store. The verification procedure is: (1) For each audit record, verify the hash chain integrity (each record includes the hash of the previous record). (2) Verify that all referenced artifacts exist and match their recorded hashes. (3) Verify that there are no gaps in the audit chain (missing records between consecutive releases). (4) Verify that all approvals are from authorized individuals and are within their authority scope. (5) Verify that all quality gate results are consistent with the verification.json artifacts. If any verification fails, the command **MUST** report the specific failure and **MUST** exit with code 1. The audit verification **MUST** be performed quarterly for active products and annually for end-of-life products within their retention period.

17.4 Audit Export

The vres audit --export command **MUST** produce a complete audit export in a format suitable for external auditors. The export **MUST** include: all audit records for the specified release range, all referenced artifacts (or their hashes with download locations), all approval records with approver identities, all quality gate results, and a manifest of the export contents. The export format **MUST** be a zip archive containing: audit-records.json (all audit records), artifacts/ (referenced artifact hashes and locations), approvals.json, quality-gates.json, and export-manifest.json. The export **MUST** be signed with the VVU audit export key to enable external auditors to verify the export integrity. The export **MUST** be available within 5 business days of an audit request.

Volume XVI

Research Validation

Chapter 18 — Research Validation Requirements

18.1 HBK Mk-II Hydraulic Simulation Validation

HBK Mk-II operates in South African municipal water infrastructure. Validation requirements are concrete and safety-relevant — hydraulic simulation results inform decisions about physical infrastructure where errors could affect public health. The following acceptance criteria **MUST** be satisfied for every HBK Mk-II release:

Criterion	Threshold	Req.	Rationale
Brier Score	< 0.02	MUST	Prediction accuracy; >0.02 triggers TRIP verdict
MCMC convergence	R-hat < 1.01	MUST	Gelman-Rubin diagnostic for chain convergence
Mass conservation	< 0.1% error	MUST	Conservation of mass in hydraulic simulation
Energy conservation	< 0.5% error	MUST	Conservation of energy in hydraulic simulation
Pressure range	Within sensor spec	MUST	0-600 kPa for municipal water
Flow rate range	Within sensor spec	MUST	0-500 L/min for research flows
Sensor calibration	Within tolerance	MUST	Per-sensor calibration certificate on file
Temperature range	0-45 C operating	MUST	South African climate range
Simulation timestep	Stable per CFL	MUST	Courant-Friedrichs-Lewy condition satisfied
Statistical confidence	>= 95% CI	MUST	Minimum confidence interval for published results

18.2 ProofBridge Validation

ProofBridge issues Ed25519-signed receipts anchored into Merkle Mountain Ranges (MMR). Validation **MUST** verify: (a) receipt signature integrity (Ed25519 verify returns true), (b) MMR append-only property (no deletions or modifications to historical peaks), (c) receipt content matches the claimed transaction (amount, recipient, timestamp), (d) ZK-proof generation produces a valid proof that verifies against the verification key, and (e) ledger consistency (MMR root hash matches the published root for the block). These validations **MUST** be performed for every release and **MUST** be included in the evidence package as part of the test-report.json artifact.

18.3 IVE Validation

IVE validation **MUST** verify the core evidence pipeline: (a) frozen contract schema validation (results.json matches the frozen schema with all required fields present), (b) adapter attribution integrity (every normalized field has a source attribution, no fabricated fields), (c) pipeline preservation (historical run records are byte-for-byte unchanged), (d) checksum spec satisfaction (all 7 rules from CHECKSUM_SPEC satisfied), and (e) evaluation disposition accuracy (GO/NO-GO correctly reflects the actual gate results). The zero fabrication rule is absolute: any missing value **MUST** be explicitly marked as UNDEFINED, MISSING, REQUIRES VALIDATION, or PENDING — never inferred or defaulted.

18.4 Evidence Packages

Research releases **MUST** include an evidence package containing: (a) the simulation or experimental data, (b) the analysis code with exact version pinning, (c) the environment specification (hardware, OS, runtime versions), (d) the validation results, (e) the statistical analysis with confidence intervals, and (f) a reproducibility statement attesting that an independent researcher has reproduced the results. Evidence packages **MUST** be cryptographically sealed (SHA-256 manifest + Ed25519 signature) to prevent tampering after publication. Evidence packages for academic publication **MUST** conform to the DORA data sharing requirements (Section 12.4).

18.1.1 Brier Score Threshold Derivation

The Brier Score threshold of 0.02 for HBK Mk-II is derived from the TRIP (Testing, Review, and Improvement Process) requirements for municipal water infrastructure in South Africa. The Brier Score measures the mean squared difference between predicted probabilities and actual outcomes. A perfect prediction has a Brier Score of 0, while a random prediction has an expected Brier Score of 0.25 for binary outcomes. The threshold of 0.02 corresponds to a prediction that is correct approximately 86% of the time for a balanced binary event. This threshold was selected because: (a) it is significantly better than the no-skill baseline (Brier Score 0.25), (b) it is achievable with the current HBK Mk-II model architecture and available sensor data, and (c) it provides sufficient predictive accuracy for operational decisions in municipal water infrastructure. The threshold **MUST** be reviewed annually and **MAY** be tightened as model capability improves but **MUST NOT** be relaxed without Security Officer approval and documented justification.

18.1.2 MCMC Convergence Diagnostics

Markov Chain Monte Carlo (MCMC) convergence **MUST** be verified using the Gelman-Rubin diagnostic (R-hat) before any inference results are included in a release. The R-hat statistic compares the variance between chains to the variance within chains. An R-hat value of 1.0 indicates perfect convergence, while values above 1.01 indicate that the chains have not converged to the same stationary distribution. The VRES requirement of R-hat < 1.01 is the standard threshold used in Bayesian statistics for declaring convergence. In addition to R-hat, the following diagnostics **MUST** be checked: (a) effective sample size (ESS) > 400 per chain, ensuring sufficient samples for stable inference; (b) no divergent transitions in Hamiltonian Monte Carlo, indicating that the sampler is not encountering regions of high curvature; (c) energy Bayesian fraction of missing information (E-BFMI) > 0.3, indicating adequate exploration of the energy distribution. All diagnostic values **MUST** be included in the evidence package.

18.2.1 ProofBridge Evidence Package Specification

The ProofBridge evidence package for a release **MUST** contain: (a) A set of test receipts with known values, generated by the release candidate and verified against the expected values. (b) MMR state verification showing that the append-only property holds for all test receipts. (c) ZK-proof verification records showing that generated proofs verify against the circuit verification key. (d) Receipt signature verification records showing that each test receipt has a valid Ed25519 signature. (e) Ledger consistency verification showing that the MMR root hash matches the on-chain commitment. The evidence package **MUST** be generated automatically by the CI/CD pipeline and **MUST** be reviewed by the Security Officer before release. The evidence package **MUST** be included in the enterprise procurement pack (Volume XVII) for external audit.

18.4.1 Peer Review Requirements

Research releases (publications, simulation results, experimental data) **MUST** undergo peer review before publication. The peer review requirements are: (a) The reviewer **MUST** be an independent researcher who was not involved in the research or the software development. (b) The reviewer **MUST** successfully reproduce the published results using the reproducibility package (Section 12.4.1). (c) The reviewer **MUST** verify that the statistical analysis is correct and that the confidence intervals are valid. (d) The reviewer **MUST** document any discrepancies between the published results and their reproduction attempt. (e) The review **MUST** be documented in the evidence package with the reviewer identity, date, and result. For HBK Mk-II publications, the reviewer **SHOULD** have domain expertise in hydraulic engineering or Bayesian statistics. The peer review requirement is separate from and in addition to the code review process (Volume II, Section 4.2).

Volume XVII

Enterprise Procurement Pack

Chapter 19 — Procurement Deliverables

19.1 Pack Contents

The enterprise procurement pack is automatically generated for every major and minor release. It is the primary deliverable for enterprise customers, municipalities, universities, and government procurement teams. The pack **MUST** contain the following documents, each generated from the release artifacts and product metadata using the vres release --procurement command.

Document	Content	Format
Executive Summary	High-level product overview, key capabilities, business value	PDF
Architecture Overview	System architecture diagram, component inventory, integration points	PDF
Security Overview	Security model, threat landscape, controls, incident response	PDF
Compliance Matrix	Regulatory framework mapping, control coverage, gap analysis	PDF + XLSX
SBOM	Software bill of materials (SPDX + CycloneDX)	JSON
Licensing	License terms, dependency licenses, obligations, restrictions	PDF
Pricing (ZAR)	Proudly South African pricing in South African Rand	PDF
Support Matrix	Support tiers, SLAs, escalation paths, coverage hours (SAST)	PDF
Risk Register	Identified risks, likelihood, impact, mitigations	PDF + XLSX
Implementation Guide	Step-by-step deployment, configuration, and validation	PDF

19.2 Proudly South African Pricing

All pricing in the procurement pack **MUST** be denominated in South African Rand (ZAR). The pricing model **MUST** reflect the Proudly South African values: (a) pricing **MUST** be transparent with no hidden fees or undisclosed costs, (b) community and academic licensing **MUST** be available at reduced rates for South African institutions, (c) stokvel and cooperative pricing models **MUST** be supported for community organizations (Ubuntu Pools integration), (d) government pricing **MUST** comply with the Preferential Procurement Policy Framework Act (Act 5 of 2000) and B-BBEE requirements, and (e) all prices **MUST** include VAT at the current South African rate (15%). The pricing section **MUST** include a total cost of ownership calculation covering license, support, infrastructure, and training over a 3-year period.

19.1.1 Executive Summary Template

The executive summary **MUST** contain: (a) Product name and version. (b) One-paragraph product description targeting a non-technical executive audience. (c) Key capabilities as a bulleted list (maximum 10 items). (d) Business value proposition specific to the target sector (municipal water, financial services, research). (e) Competitive differentiators, with emphasis on the VRES advantages: reproducible builds, cryptographic verification, and auditability. (f) Deployment options summary with topology names and key characteristics. (g) Pricing summary in ZAR (Section 24.3). (h) Support options summary. (i) Compliance and certification status. (j) Proudly South African membership and B-BBEE status. The executive summary **MUST** be no longer than 2 pages and **MUST** be suitable for inclusion in a procurement committee agenda.

19.2.1 Pricing Table Structure

The pricing table **MUST** be structured as follows, with all amounts in ZAR including VAT.

Tier	Monthly (incl. VAT)	3-Year TCO	License	Support
Community	Free	Free	AGPL-3.0-or-later	Community forum
Academic	R 0	R 0	AGPL-3.0-or-later	Academic support (SA institutions only)
Starter	R 4,500/mo	R 145,800	Commercial	Email support, business hours (SAST)
Professional	R 15,000/mo	R 486,000	Commercial	Priority support, extended hours
Enterprise	R 45,000/mo	R 1,458,000	Commercial	Dedicated support, 24/7, SLA
Government	R 25,000/mo	R 810,000	Commercial	Compliance support, PFMA/MFMA
Municipal	R 12,000/mo	R 388,800	Commercial	Water infrastructure specialist support

19.2.2 Support Tier Details

Each support tier has defined service level agreements (SLAs) aligned with South African Standard Time (SAST, UTC+2). The following table defines the support tiers and their SLAs.

Tier	Response Time	Channels	Coverage Hours	Days	Target
Community	Best effort	Forum only	N/A	N/A	N/A
Academic	Next business day	Email	SAST 08:00-17:00	Mon-Fri	SA universities
Starter	4 business hours	Email + chat	SAST 08:00-18:00	Mon-Fri	Small teams

Tier	Response Time	Channels	Coverage Hours	Days	Target
Professional	2 business hours	Email + chat + phone	SAST 07:00-21:00	Mon-Sat	Growing teams
Enterprise	15 minutes	All channels + dedicated	24/7/365	Always	Large orgs
Government	1 hour	All + compliance	SAST 07:00-22:00	Mon-Sat	Gov/municipal

Volume XVIII

Disaster Recovery

Chapter 20 — Disaster Recovery Requirements

20.1 Recovery Objectives

Every VVU product deployment **MUST** define Recovery Time Objective (RTO) and Recovery Point Objective (RPO). RTO is the maximum acceptable time to restore service after a failure. RPO is the maximum acceptable data loss measured in time. These objectives **MUST** be defined per deployment topology (Volume XI) and **MUST** be validated through periodic restore drills.

Topology	RTO	RPO	Recovery Method
Single Node	4 hours	1 hour	Manual restore from backup
High Availability	15 minutes	5 minutes	Automatic failover
Air-Gapped	8 hours	4 hours	Manual restore from offline media
Government	1 hour	15 minutes	Pre-staged failover + audit recovery
Research Cluster	2 hours	30 minutes	Cluster rebuild + data restore
Cloud	5 minutes	1 minute	Multi-AZ failover
HBK Appliance	30 minutes	5 minutes	Hot standby + sensor cache
IVE Appliance	15 minutes	5 minutes	Active-passive + evidence lock

20.2 Backup and Restore

Backups **MUST** be performed according to the RPO schedule. Backup verification **MUST** be performed weekly by restoring to a test environment and validating data integrity. Restore drills **MUST** be performed quarterly and **MUST** document: the drill start time, the restore completion time, the data integrity verification result, any issues encountered, and remediation actions. Key escrow **MUST** be implemented for signing keys: the product **MUST** be able to verify historical release signatures even if the current signing key is lost, by retaining retired keys in an escrow system with threshold access.

20.1.1 Per-Topology DR Procedures

Each deployment topology (Volume XI) has a specific disaster recovery procedure. The procedures are designed to meet the RTO and RPO objectives defined in Section 20.1. For the High Availability topology: the primary failure mode is node failure. Automatic failover to the standby node occurs within 15 minutes. Data replication ensures RPO of 5 minutes. The procedure is tested monthly by terminating a node and measuring recovery time. For the Air-Gapped topology: recovery requires physical access to the deployment site. The procedure is: (1) Identify the failed component. (2) Obtain the backup media (stored offsite in a fireproof safe). (3) Restore the application from backup. (4) Verify data integrity using checksums. (5) Resume operations. The 8-hour RTO accounts for physical access time. For the Government topology: recovery is supported by pre-staged failover infrastructure in a separate security zone. The 1-hour RTO is achieved through automated failover with audit log recovery from the Trust Sphere.

20.2.1 Backup Verification Procedure

Backup verification **MUST** be performed weekly using the following procedure: (1) Select the most recent backup. (2) Restore the backup to a dedicated test environment (not production). (3) Run the application health check suite against the restored environment. (4) Verify data integrity by comparing row counts, checksums, and key business metrics against the production values at the backup timestamp. (5) Verify that the restored application can serve requests correctly. (6) Record the verification result in the audit trail. (7) Destroy the test environment to prevent data leakage. If the backup verification fails, the incident **MUST** be escalated to the Release Engineer and the backup **MUST** be re-created. Failed backup verifications **MUST** be resolved within 24 hours — an unverified backup is effectively no backup.

20.2.2 Restore Drill Template

Restore drills **MUST** be performed quarterly and **MUST** follow this template: (a) Drill identifier and date. (b) Topology being tested. (c) RTO and RPO objectives. (d) Actual recovery time (stopwatch from drill start to service restoration). (e) Data loss measurement (comparison of data at drill start vs. data after recovery). (f) Issues encountered during the drill. (g) Remediation actions for any issues. (h) Updated RTO/RPO estimates based on drill results. (i) Sign-off from the Release Engineer. The drill results **MUST** be recorded in the audit trail. If the drill reveals that RTO or RPO objectives cannot be met, the deployment **MUST** be re-architected or the objectives **MUST** be revised with documented justification and stakeholder approval.

Volume XIX

Product-Specific Extensions

Chapter 21 — Product Extension Model

21.1 Common Core + Product Overlays

VRES follows a common-core-plus-overlays architecture. The common core (Volumes I-XVIII) applies to all VVU products. Each product defines an overlay that extends or constrains the common core for its domain requirements. The overlay **MUST NOT** weaken any common core requirement — it **MAY** only add requirements or tighten thresholds. Future VVU products inherit the common core baseline and extend only where needed, ensuring that the release engineering standard remains consistent across the product portfolio.

21.2 IVE Overlay

The IVE product overlay adds the following requirements: (a) evidence pipeline integrity — every artifact in the evidence pipeline **MUST** have source attribution and **MUST NOT** contain fabricated fields, (b) interactive verification — the verification workflow **MUST** support interactive evidence inspection and challenge, (c) dashboard completeness — all 22 IVE panels **MUST** be present and functional in release builds, (d) frozen contract stability — the results.json frozen contract schema **MUST NOT** change between patch releases, and (e) license status — the license_status field in results.json **MUST** be VALIDATED for any release to proceed past the Verification state.

21.3 HBK Mk-II Overlay

The HBK Mk-II product overlay adds the following requirements: (a) hardware calibration — sensor calibration certificates **MUST** be current and on file for every deployed sensor, (b) firmware signing — Tier 1 firmware **MUST** be signed with an HSM-backed key with threshold custody, (c) hardware-in-the-loop testing — HIL tests **MUST** pass before any firmware release (Section 8.2), (d) operating context — the South African municipal water infrastructure operating context **MUST** be documented with all regulatory approvals tracked (REQUIRES VALIDATION until all approvals obtained), and (e) Brier Score threshold — the TRIP threshold of 0.02 **MUST NOT** be exceeded in any released prediction model.

21.4 ProofBridge Overlay

The ProofBridge product overlay adds the following requirements: (a) transaction integrity — every receipt **MUST** be Ed25519-signed and anchored in the MMR, (b) safety kernel — the receipt issuance kernel **MUST** be isolated from the application layer and **MUST** fail closed (deny receipt issuance) on any integrity check failure, (c) ledger validation — the MMR root hash **MUST** be verified against the on-chain commitment on every startup and at configurable intervals, and (d) ZK-proof correctness — generated ZK proofs **MUST** verify against the circuit verification key.

21.1.1 Overlay Specification Format

Each product overlay **MUST** be specified in a vres-overlay.yaml file in the product directory. The overlay file defines: (a) additional requirements beyond the common core, (b) tightened thresholds for existing requirements, (c) product-specific quality gates, and (d) product-specific artifacts. The overlay format is YAML with a defined schema. The vres lint command **MUST** validate the overlay file against the schema and **MUST** verify that the overlay does not weaken any common core requirement. An overlay that weakens a common core requirement **MUST** be reported as a conformance violation.

```
# vres-overlay.yaml for HBK Mk-II
product: hbk-mkii
version: "1.0.0"

# Tightened thresholds
thresholds:
unit_coverage: ">= 90%" # Common core: >= 80%
integration_coverage: ">= 85%" # Common core: >= 70%

# Additional quality gates
additional_gates:
- id: G20
name: "HIL Test"
criteria: "All HIL tests pass"
required: true
evaluator: "automated"
- id: G21
name: "Brier Score"
```

```
criteria: "Brier Score < 0.02"
required: true
evaluator: "automated"

# Product-specific artifacts
additional_artifacts:
- name: "calibration-certificates"
format: "PDF"
required: true
- name: "hil-test-report"
format: "JSON"
required: true
```

21.2.1 IVE Overlay Details

The IVE product overlay adds the following specific requirements beyond the common core: (a) Evidence pipeline integrity — every artifact in the evidence pipeline **MUST** have source attribution (the adapter and field that produced the value) and **MUST NOT** contain fabricated fields (Section 1.2.5.1). (b) Interactive verification — the verification workflow **MUST** support interactive evidence inspection and challenge, where a user can drill into any evaluation result to see the source evidence and challenge the result. (c) Dashboard completeness — all 22 IVE panels **MUST** be present and functional in release builds. The panel inventory **MUST** be checked by an automated test. (d) Frozen contract stability — the results.json frozen contract schema **MUST NOT** change between patch releases. Schema changes **MUST** only occur in minor or major releases with a migration guide. (e) License status — the license_status field in results.json **MUST** be VALIDATED for any release to proceed past the Verification state. This field is the gate that connects the dual-license implementation (Section 24.1) to the release process.

21.5 Future Product Onboarding Guide

When a new VVU product is created, it inherits the VRES common core and must define a product overlay. The onboarding procedure is: (1) Create the product directory in the monorepo (src/new-product/). (2) Create the vres-overlay.yaml file specifying product-specific requirements. (3) Run vres init --product=new-product to generate the product configuration, templates, and CODEOWNERS entries. (4) Implement the product with the common core requirements. (5) Run vres doctor to verify the product meets all common core requirements. (6) Run vres lint to verify the product meets its overlay requirements. (7) Add the product to the CI/CD pipeline configuration. (8) Document the product in the VRES volume XIX update. Future products **MUST NOT** weaken common core requirements — they **MAY** only add or tighten. The onboarding process ensures that new products start with the full VRES baseline and extend only where their domain requires.

Volume XX

Certification Readiness

Chapter 22 — External Framework Mapping

22.1 Purpose

This volume maps VRES requirements to external frameworks to facilitate future certification. The goal is **NOT** to claim certification — it is to make certification work substantially easier by demonstrating that VRES already addresses the controls required by these frameworks. When a certification body audits a VVU product, the VRES-to-framework mapping provides traceability from each control to the specific VRES requirement that satisfies it, reducing the evidence collection burden.

22.2 NIST Secure Software Development Framework (SP 800-218)

The NIST SSDF defines practices for secure software development. The following VRES volumes map to SSDF practices: PO.1 (Define roles) → Volume I Chapter 3; PO.2 (Implement roles) → Volume II CODEOWNERS; PO.3 (Arch for security) → Volume V; PO.4 (Maintain secure env) → Volume III hermetic builds; PO.5 (Define security expectations) → Volume V; PS.1 (Protect code) → Volume V secret detection + signing; PS.2 (Verify third-party) → Volume IV SBOM + license audit; PW.1 (Design to meet security) → Volume V; PW.2 (Review design) → Volume II review requirements; PW.4 (Review code) → Volume II CODEOWNERS; PW.5 (Test code) → Volume VI; PW.6 (Configure build) → Volume III; PW.7 (Review test) → Volume XIII quality gates; PW.8 (Archive release) → Volume XV audit; PW.9 (Publish release) → Volume IX artifacts; RV.1 (Identify vulnerabilities) → Volume IV CVE scanning; RV.2 (Assess vulnerabilities) → Volume V security analysis; RV.3 (Remediate) → Volume XII hotfix strategy.

22.3 SLSA Levels

VRES maps to SLSA as follows: SLSA Level 1 (provenance exists) — satisfied by Volume VII provenance.json. SLSA Level 2 (hosted build platform) — satisfied by Volume XIV CI/CD pipelines on hosted platforms. SLSA Level 3 (hardened build platform) — satisfied by Volume III hermetic builds + Volume V secret detection. SLSA Level 4 (two-party review) — target for HBK Mk-II Tier 1 firmware, satisfied by Volume II CODEOWNERS with two required reviewers. All VVU production releases **MUST** achieve SLSA Level 3. Safety-critical components **MUST** target SLSA Level 4.

22.4 ISO/IEC 27001 Supporting Controls

VRES supports the following ISO 27001 controls: A.8.1.1 (Asset inventory) → Volume IX release artifacts; A.8.9 (Configuration management) → Volume III build engineering; A.8.25 (Secure development lifecycle) → Volumes I-V; A.8.26 (Application security requirements) → Volume V; A.8.28 (Secure coding) → Volume VI testing; A.8.30 (Secure testing) → Volume VI security testing; A.8.31 (Secure development) → Volume I philosophy + Volume V security; A.8.32 (Change management) → Volume II branch strategy; A.8.33 (Test information) → Volume VI test results; A.8.34 (Attack surface) → Volume V security analysis.

22.5 IEC 61508 Principles

For safety-related systems (HBK Mk-II Tier 1), VRES addresses IEC 61508 principles: Functional safety management → Volume I governance + Volume XIII quality gates; Safety requirements specification → Volume XVI HBK criteria; Verification → Volume VII + Volume XVI validation; Validation → Volume XVI HIL testing; Functional safety assessment → Volume XIII quality gates; Configuration management → Volume III + Volume IX; Software development → Volumes II-VI. VRES does not claim IEC 61508 compliance but provides the organizational and technical controls that would be required for a SIL assessment, reducing the gap analysis for future certification.

22.2.1 NIST SSDF Detailed Control Mapping

The following detailed mapping shows how each NIST SSDF practice and task is addressed by specific VRES requirements. This mapping enables an auditor to trace from each SSDF control to the VRES requirement that satisfies it, reducing the evidence collection burden for certification.

SSDF Practice	Description	VRES Mapping
PO.1.1	Define security roles	VRES Ch 3: Role definitions, authority, separation of duties
PO.1.2	Specify role responsibilities	VRES Ch 3: Role responsibility table, prohibited actions
PO.2.1	Implement roles	VRES Vol II: CODEOWNERS, protected branches
PO.2.2	Review role compliance	VRES Vol XV: Audit records, approval tracking
PO.3.1	Security architecture	VRES Vol V: Security requirements, threat model
PO.3.2	Security design review	VRES Vol V: SAST/DAST, security gate G08
PO.4.1	Secure build environment	VRES Vol III: Hermetic builds, offline capability
PO.4.2	Secure CI/CD	VRES Vol XIV: Pipeline security, runner hardening
PO.5.1	Security expectations	VRES Vol V: Incident response, key management
PO.5.2	Compliance requirements	VRES Vol X: License, export, privacy compliance

22.3.1 SLSA Level Detailed Mapping

SLSA Level 1 (Provenance exists): Satisfied by Volume VII provenance.json, which records the builder identity, source commit, build configuration, and output artifact hashes. The provenance is generated automatically by the CI/CD pipeline (Volume XIV). SLSA Level 2 (Hosted build platform): Satisfied by Volume XIV CI/CD pipelines running on GitHub Actions (hosted runners) or GitLab CI (shared runners). The hosted platform provides isolation between builds and prevents build-parameter tampering. SLSA Level 3 (Hardened build platform): Satisfied by Volume III hermetic builds (no host dependencies, network restricted to declared repositories), Volume V secret detection (no secrets in build environment), and Sigstore keyless signing (no persistent signing keys in the build environment). SLSA Level 4 (Two-party review): Target for HBK Mk-II Tier 1 firmware, satisfied by Volume II CODEOWNERS requiring two reviewers from different teams for all changes to protected directories.

22.5.1 IEC 61508 Gap Analysis

VRES does not claim IEC 61508 compliance but provides the organizational and technical controls that reduce the gap for future SIL assessment. The following gap analysis identifies what VRES addresses and what additional work would be required for IEC 61508 SIL 2 compliance. Addressed by VRES: functional safety management (Vol I governance), software development lifecycle (Vol I-II lifecycle), verification (Vol VII, Vol XIII gates), configuration management (Vol III build, Vol IX artifacts), and software maintenance (Vol XII operations). Gaps requiring additional work: formal methods for safety requirements specification, independent functional safety assessment by an accredited assessor, systematic hazard analysis (HAZOP, FMEA) documented to IEC 61508 requirements, safety validation against the safety requirements specification, and demonstrated compliance with IEC 61508-3 software safety integrity requirements for the target SIL. The gap analysis **MUST** be reviewed annually and **MUST** be included in the enterprise procurement pack for HBK Mk-II customers.

Volume —

Supporting Tooling: vres CLI

Chapter 23 — vres Command-Line Interface

23.1 Interface Contract

The vres CLI is the reference implementation of VRES. It provides both human-readable reports and machine-readable JSON output for CI/CD integration. Every subcommand **MUST** support the --format json flag for machine-readable output and **MUST** exit with code 0 on success, 1 on failure, and 2 on usage error. The vres CLI **MUST** be implemented in Python 3.12+ with no mandatory dependencies beyond the Python standard library (optional dependencies for enhanced functionality are allowed). The CLI **MUST** be installable via pip and **MUST** be available as vres on the PATH after installation.

23.2 Subcommand Specifications

Command	Purpose	Behavior	Exit Codes
vres init	Initialize VRES in a repository	Creates .vres/ config directory, templates, hooks	0=created, 1=exists
vres doctor	Validate repository health	Checks all VRES requirements, reports status	0=healthy, 1=issues
vres lint	Lint against VRES standard	Checks conformance level, reports violations	0=conformant, 1=violations
vres test	Execute VRES test suite	Runs required test categories, generates report	0=pass, 1=fail
vres sbom	Generate SBOM	Produces SPDX + CycloneDX from lockfiles	0=generated, 1=error
vres manifest	Generate cryptographic manifest	SHA-256 index of all release artifacts	0=generated, 1=error
vres sign	Sign release artifacts	Ed25519 signing via Sigstore/Cosign	0=signed, 1=error
vres verify	Verify release integrity	Manifest + signature + provenance verification	0=verified, 1=error
vres release	Execute release process	Orchestrates lifecycle from candidate to release	0=released, 1=blocked
vres audit	Generate audit record	Creates immutable audit entry for the release	0=created, 1=error
vres rollback	Execute rollback procedure	Restores previous version, generates report	0=rolled back, 1=error
vres archive	Archive release artifacts	Moves release to long-term storage with integrity	0=archived, 1=error

23.3 Reference Implementation: vres doctor

The vres doctor command **MUST** check the following: (1) repository structure matches Volume II requirements, (2) CODEOWNERS file exists and references valid teams, (3) protected branches configured correctly, (4) CI/CD pipeline implements all quality gates, (5) license file exists and is valid, (6) lockfile exists and is consistent with installed dependencies, (7) no known critical or high CVEs in dependencies, (8) no secrets detected in repository, (9) SBOM can be generated, (10) signing key or Sigstore configuration is available. Each check **MUST** report PASS, FAIL, or WARN with a specific message. The --offline flag **MUST** verify that all dependencies are available locally for air-gapped builds.

23.4 Output Schema

Machine-readable output (--format json) **MUST** conform to the following schema for all commands:

```
{
"command": "vres doctor",
"version": "1.0.0",
"timestamp": "2026-08-06T12:00:00Z",
"result": "PASS|FAIL|WARN",
"checks": [
{
"id": "repo-structure",
"name": "Repository Structure",
"result": "PASS",
"message": "All required directories present",
"severity": null
}
],
"summary": {
"total": 10,
"pass": 8,
"fail": 1,
"warn": 1
}
}
```

23.3.1 Reference Implementation: vres init

The vres init command initializes VRES configuration in a repository. The command MUST: (1) Create the .vres/ directory with configuration files. (2) Copy the CODEOWNERS template (Section 4.5.1) to the repository root if one does not exist. (3) Copy the PR template (Section 4.6.1) to .github/pull_request_template.md. (4) Copy the issue templates to .github/ISSUE_TEMPLATE/. (5) Create the .vres/vres.yaml configuration file with product defaults. (6) Add pre-commit hook configuration for secret scanning and formatting. (7) Verify that the repository structure matches the monorepo standard (Section 4.1). The command **MUST** be idempotent — running it multiple times **MUST NOT** overwrite existing configuration. The --force flag **MAY** be used to overwrite, but **MUST** prompt for confirmation.

```
# vres init usage
vres init # Initialize with defaults
vres init --product=ive # Initialize for IVE product
vres init --force # Overwrite existing configuration
vres init --offline # Configure for offline/air-gapped use

# Creates the following structure:
.vres/
vres.yaml # VRES configuration
gates.yaml # Quality gate configuration
products/
```

```
ive.yaml # IVE product overlay
templates/
pr.md # PR template
release.md # Release template
```

23.3.2 Reference Implementation: vres sbom

The vres sbom command generates SBOM documents from the package manager lockfiles. The command MUST: (1) Detect the package managers in use (pip/uv, cargo, npm/bun, go, conan). (2) Extract dependency information from each lockfile, including direct and transitive dependencies. (3) Resolve license information for each dependency from the package metadata and the SPDX License List. (4) Generate the SPDX SBOM (sbom.spdx.json) conforming to SPDX 2.3. (5) Generate the CycloneDX SBOM (sbom.cyclonedx.json) conforming to CycloneDX 1.5. (6) Validate both SBOMs against their respective schemas. (7) Verify that both SBOMs contain the same dependency set. The --verify flag **MUST** validate the SBOM against the actual installed dependencies and **MUST** report any discrepancies.

23.3.3 Reference Implementation: vres release

The vres release command orchestrates the release lifecycle from candidate to release. The command MUST: (1) Verify that the source tree is at the expected commit and tag. (2) Execute the quality gate pipeline (G01-G19). (3) Generate all release artifacts (Volume IX). (4) Sign all artifacts (Section 7.3). (5) Verify all signed artifacts (Section 9.2). (6) Generate the audit record (Volume XV). (7) Publish the release to the staging repository. (8) Optionally generate the enterprise procurement pack (--procurement flag). The --dry-run flag **MUST** execute all steps without publishing, producing a preview of the release. The --hotfix flag **MUST** use the abbreviated lifecycle (Section 14.4). The release command **MUST** be interactive — it **MUST** prompt for confirmation before publishing to production, and the prompt **MUST** show a summary of the release artifacts, quality gate results, and any warnings.

23.5 Test Vectors

The vres CLI **MUST** include test vectors that verify the correctness of each subcommand. The test vectors are: (a) A known-good repository with expected vres doctor output. (b) A known-good SBOM with expected SPDX and CycloneDX output. (c) A known-good manifest with expected SHA-256 hashes. (d) A known-good signature with expected verification result. (e) A known-bad repository with expected error output for each violation type. The test vectors **MUST** be included in the vres package and **MUST** be executed by vres test --vectors. The test vectors **MUST** be updated when the VRES standard is updated to ensure that the CLI remains consistent with the standard.

Volume —

Normative Blocker Resolution

Chapter 24 — Blockers for license_status = VALIDATED

24.1 Dual-License Stack

The current VVU release disposition is NO-GO due to a missing license. This section defines the dual-license stack as a normative requirement that, when implemented, resolves this blocker. VVU products **MUST** implement a dual-license model with the following structure: (a) Open-source track: AGPL-3.0-or-later — requires source disclosure for network use, encourages contribution, provides community licensing. (b) Enterprise track: VVU Commercial License — permits proprietary integration, includes warranty and indemnification, provides enterprise support entitlement. The LICENSE file at the repository root **MUST** contain the AGPL-3.0-or-later text. The COMMERCIAL-LICENSE.md file **MUST** describe the commercial terms, pricing (in ZAR, Section 24.3), and how to obtain a commercial license. A contributing entity (individual or organization) **MUST** sign a Contributor License Agreement (CLA) that grants VVU the right to dual-license their contribution under both tracks. The CLA **MUST** be managed through a CLA assistant that checks contributor status on every pull request.

24.2 Integrity-Gate Script

The integrity-gate script (verify_release.py) **MUST** be enhanced to implement the full VRES verification pipeline as a single entry point. The enhanced script MUST: (a) validate the results.json frozen contract against the schema (all required fields present and correctly typed), (b) validate the checksums.txt manifest against all covered artifacts (SHA-256 match), (c) validate the license status (MUST be VALIDATED, not MISSING - REQUIRES DECISION), (d) validate the SBOM exists and is parseable in both SPDX and CycloneDX formats, (e) validate digital signatures on signed artifacts, (f) produce a machine-readable verification result (exit 0 with JSON output on success, exit 1 with diagnostic JSON on failure). The script **MUST** be located at scripts/verify_release.py and **MUST** be executed as part of the CI/CD pipeline (Volume XIV) on every pull request and release candidate. The script **MUST NOT** fabricate any values — missing data **MUST** be reported as MISSING - REQUIRES DECISION.

24.3 Proudly South African ZAR Pricing Model

All VVU product pricing **MUST** be denominated in South African Rand (ZAR). This is a normative requirement, not a localization preference. The ZAR pricing model **MUST** include: (a) a published price list in ZAR for all product tiers (Community, Academic, Enterprise, Government), (b) VAT inclusion at the current South African rate (15%) with a clear breakdown of ex-VAT and inclusive prices, (c) B-BBEE level disclosure as required by the Broad-Based Black Economic Empowerment Act, (d) compliance with the Preferential Procurement Policy Framework Act (Act 5 of 2000) for government procurement, (e) support for South African payment methods: EFT, Stitch instant EFT, card payment, and purchase order for government and institutional buyers, (f) Ubuntu Pools (stokvel) integration with contribution ranges in ZAR (R 500 - R 5,000 / month) and ProofBridge receipt for every contribution, (g) a total cost of ownership model covering license, support, training, and infrastructure over 1, 3, and 5 year periods, and (h) Proudly South African membership and branding on all procurement materials.

24.4 Blocker Resolution Sequence

The three blockers **MUST** be resolved in the following order: (1) Dual-license stack — create LICENSE (AGPL-3.0-or-later), COMMERCIAL-LICENSE.md, and CLA configuration. (2) Integrity-gate script — enhance verify_release.py to implement the full VRES verification pipeline. (3) ZAR pricing model — publish the price list and integrate with procurement pack generation. Upon completion of all three, the license_status field in results.json **MUST** be updated from "MISSING - REQUIRES DECISION" to "VALIDATED", and the release disposition **MUST** change from NO-GO to GO WITH REQUIRED FIXES (if other non-blocker fixes remain) or GO (if all fixes are resolved). The vres doctor command **MUST** verify that all three blockers are resolved and **MUST** report the current license_status as part of its output.

24.1.1 Dual-License Implementation

The dual-license implementation **MUST** include the following artifacts and processes: (a) LICENSE file at the repository root containing the AGPL-3.0-or-later full text. (b) COMMERCIAL-LICENSE.md file describing the commercial license terms, pricing (in ZAR), and how to obtain a commercial license. (c) CONTRIBUTING.md file referencing the CLA requirement and linking to the CLA assistant. (d) CLA assistant configuration (GitHub App or CLAassistant.io) that checks contributor CLA status on every pull request. (e) License headers in all source files identifying the license and copyright holder. (f) A license compatibility check in the CI/CD pipeline that verifies no incompatible dependencies are included. The implementation **MUST** be reviewed by legal counsel and **MUST** be documented in the compliance report (Volume X).

```
# LICENSE file structure (AGPL-3.0-or-later)
# Full text of GNU Affero General Public License v3.0

# COMMERCIAL-LICENSE.md structure
# VVU Commercial License
# 1. Definitions
# 2. Grant of License
# 3. Restrictions
# 4. Warranty
# 5. Indemnification
# 6. Term and Termination
# 7. Pricing (ZAR) — see Section 24.3
# 8. Support Entitlement
# 9. Contact: licensing@vvu.org
```

24.2.1 Integrity-Gate Enhancement Implementation

The integrity-gate script (verify_release.py) **MUST** be enhanced to implement the full VRES verification pipeline. The enhanced script **MUST** include the following verification steps, each producing a PASS/FAIL result with specific diagnostic information on failure. The script **MUST** be located at scripts/verify_release.py and **MUST** be executable as part of the CI/CD pipeline. The script **MUST** produce both human-readable output (to stdout) and machine-readable output (to verification.json).

```
#!/usr/bin/env python3
# VRES Integrity Gate - verify_release.py
import json, hashlib, sys
from pathlib import Path

def verify_frozen_contract(results_path: Path) -> dict:
# Verify results.json frozen contract schema
# Check all required fields present and correctly typed
# Check license_status == 'VALIDATED'
# Check no fabricated fields (zero fabrication rule)
...

def verify_checksums(manifest_path: Path, artifacts_dir: Path) -> dict:
# Verify SHA-256 manifest against artifacts
# Parse checksums.txt
# For each entry, compute SHA-256 and compare
...

def verify_signatures(manifest_sig: Path, manifest: Path) -> dict:
# Verify Ed25519 signature of manifest
# Verify using Sigstore/Cosign
...

def verify_sbom(spdx_path: Path, cyclonedx_path: Path) -> dict:
# Verify SBOM in both formats
```

```
# Parse and validate both SBOMs
# Verify same dependency set
...

def main():
results = {
'frozen_contract': verify_frozen_contract(...),
'checksums': verify_checksums(...),
'signatures': verify_signatures(...),
'sbom': verify_sbom(...),
}
all_pass = all(r['result'] == 'PASS' for r in results.values())
json.dump(results, open('verification.json', 'w'), indent=2)
sys.exit(0 if all_pass else 1)
```

24.3.1 ZAR Pricing Table Implementation

The ZAR pricing model **MUST** be implemented as a machine-readable pricing table (pricing.json) that is included in the enterprise procurement pack. The pricing table **MUST** include: all product tiers with monthly and annual prices in ZAR, VAT calculation at the current rate (15%) with ex-VAT and inclusive breakdown, B-BBEE level, total cost of ownership over 1, 3, and 5 years including license, support, training, and infrastructure estimates, and applicable South African procurement compliance information (Preferential Procurement Policy Framework Act compliance, B-BBEE scorecard). The pricing table **MUST** be generated by the vres release --procurement command and **MUST** be reviewed by the Compliance Officer before publication. Prices **MUST** be updated when VAT rates change or when pricing policy is revised.

```
{
"schema_version": "1.0.0",
"currency": "ZAR",
"vat_rate": 0.15,
"bbbee_level": 1,
"products": {
"ive": {
"tiers": {
"community": {"monthly": 0, "annual": 0, "license": "AGPL-3.0-or-later"},
"professional": {
"monthly_ex_vat": 13043.48,
"monthly_incl_vat": 15000.00,
"annual_ex_vat": 156521.74,
"annual_incl_vat": 180000.00,
"license": "commercial",
"tco_3year": 486000.00
}
}
}
},
"payment_methods": ["eft", "stitch", "card", "purchase_order"],
"proudly_sa": true
}
```